Cryptography

Advanced Encryption Standard (AES)

Cristhian Iza

School of Mathematical and Computational Sciences
Yachay Tech University

September 9, 2026

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Introduction

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Stream Ciphers vs. Block Ciphers



UC 1. Cryptographic Foundations and Symmetric Cryptography

- Introduction to Cryptography and Security
- Classical Ciphers and Cryptanalysis
- Modular Arithmetic
- Shift and Affine Ciphers
- Symmetric Cryptography
- Stream Ciphers and basic attacks
- Block Ciphers and AES
- AES and Modes of Operation

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Bruce Schneier



Twofish,
Blowfish

Vincent Rijmen
and Joan Daemen



AES

Modern private
key encryption

Rivest, Shamir
& Aldeman



Public-k
encrypt

Ron
Rivest



Hashing

Whitfield
Diffie



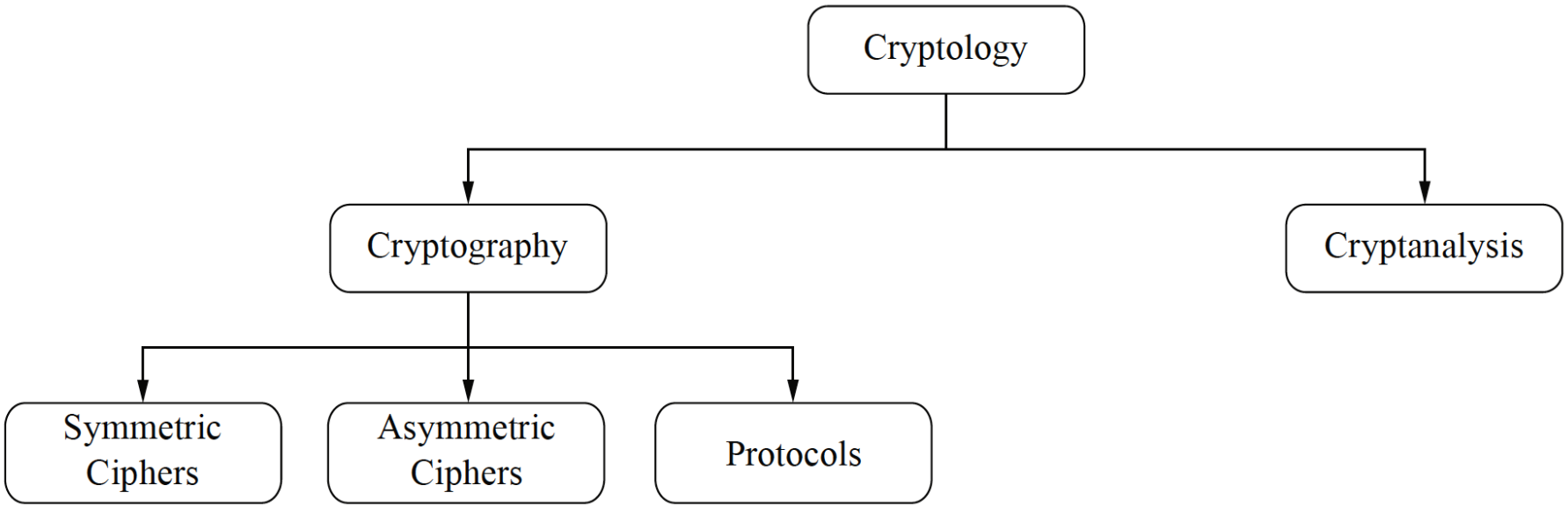
Key
interchange

Phil
Zimmerman



PGP
Encryption

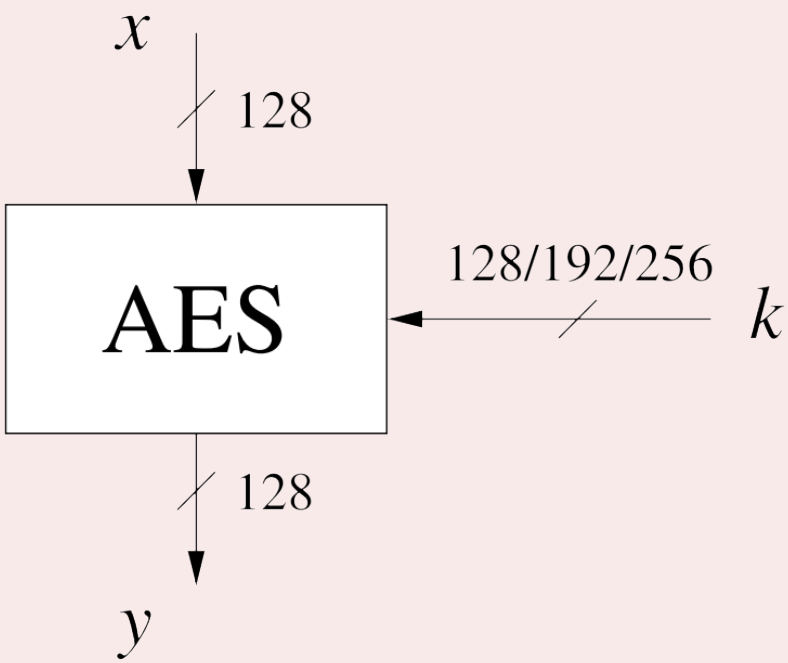
Advanced Encryption Standard (AES)



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Advanced Encryption Standard (AES)

Motivation

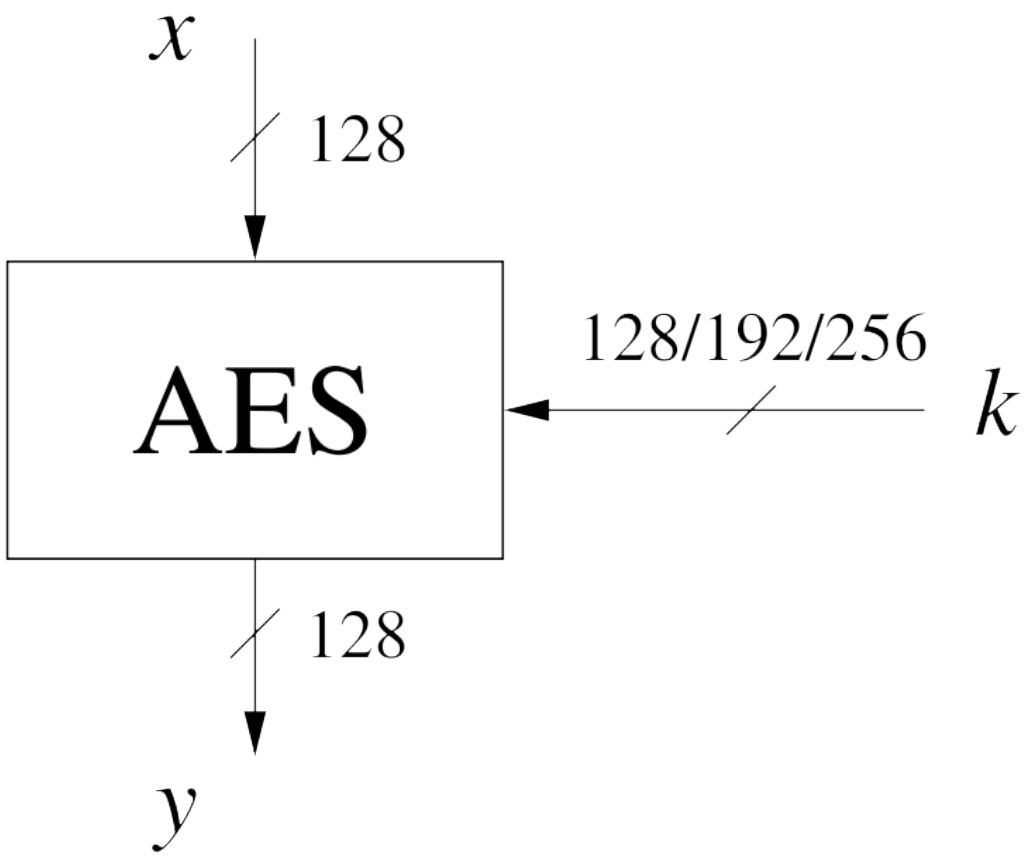


All internal opration of AES are based finite fields

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Overview of the Algorithm AES

The Rijndael algorithm, submitted for the AES, was developed by two distinguished Belgian cryptographers: Dr. Joan Daemen and Dr. Vincent Rijmen.



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Overview of the Algorithm AES

Key lengths and number of rounds for AES

key lengths	# rounds = n_r
128 bit	10
192 bit	12
256 bit	14

Remarks

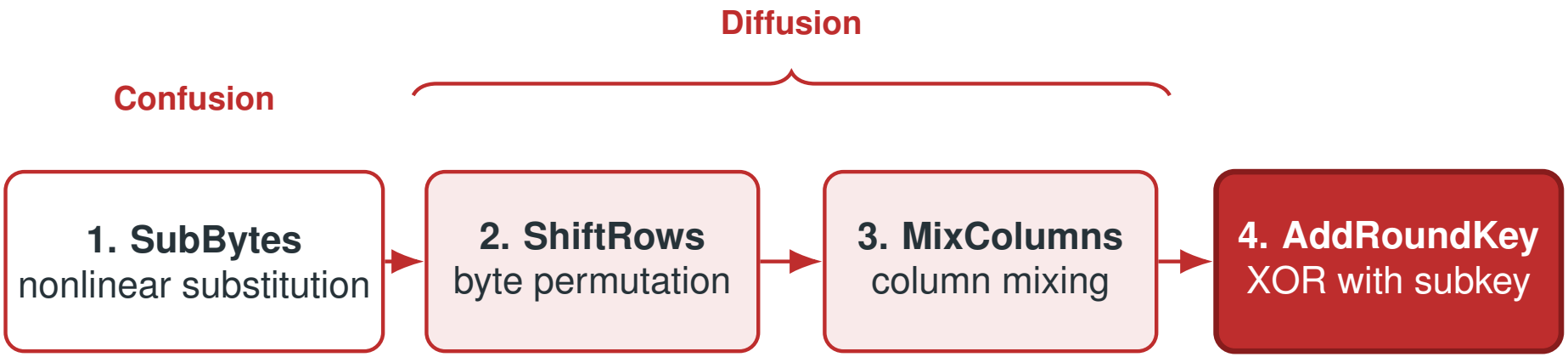
- AES is by now the most important symmetric algorithm in the world.
- NSA allows AES for classified data up to TOP SECRET with 192or 256 key lengths.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Structure of AES

AES Round

AES processes the complete **128-bit state** in every round.



The final AES round omits MixColumns.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Structure of AES (cont.)

SubBytes

Nonlinear transformation

Each byte is transformed using the AES S-Box.

$$a \longrightarrow S(a)$$

The S-Box is constructed using the **multiplicative inverse in** $\mathrm{GF}(2^8)$ followed by an affine transformation.

MixColumns

Linear transformation

Each state column is multiplied by a fixed matrix.

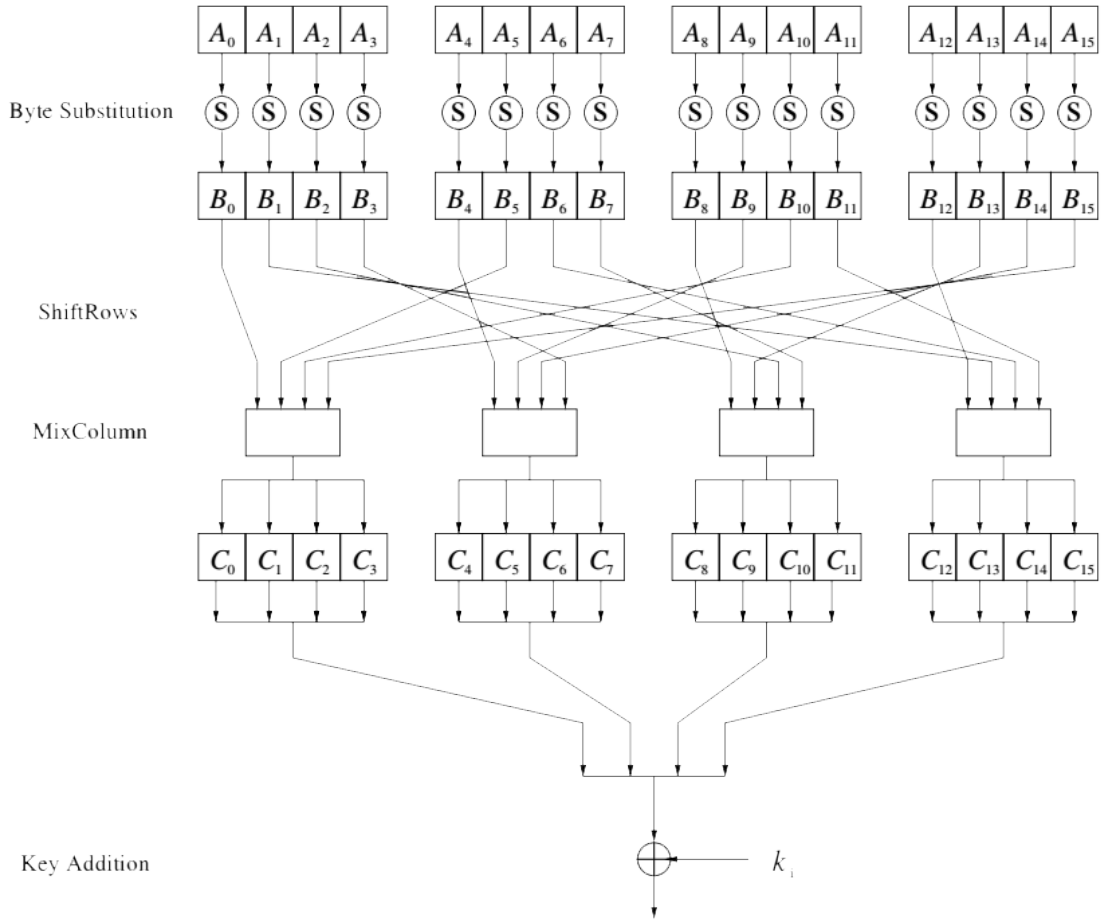
$$\mathbf{c} = \mathbf{Mb}$$

All additions and multiplications are performed in $\mathrm{GF}(2^8)$

Finite-field arithmetic provides the mathematical foundation for AES nonlinearity and diffusion.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES encrypts all 128 bits of the data path in one round



AES round function for rounds $1, 2, \ldots, n_r - 1$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Structure of AES

What occurs within an AES round?

AES transforms the entire **128-bit state** through four operations:

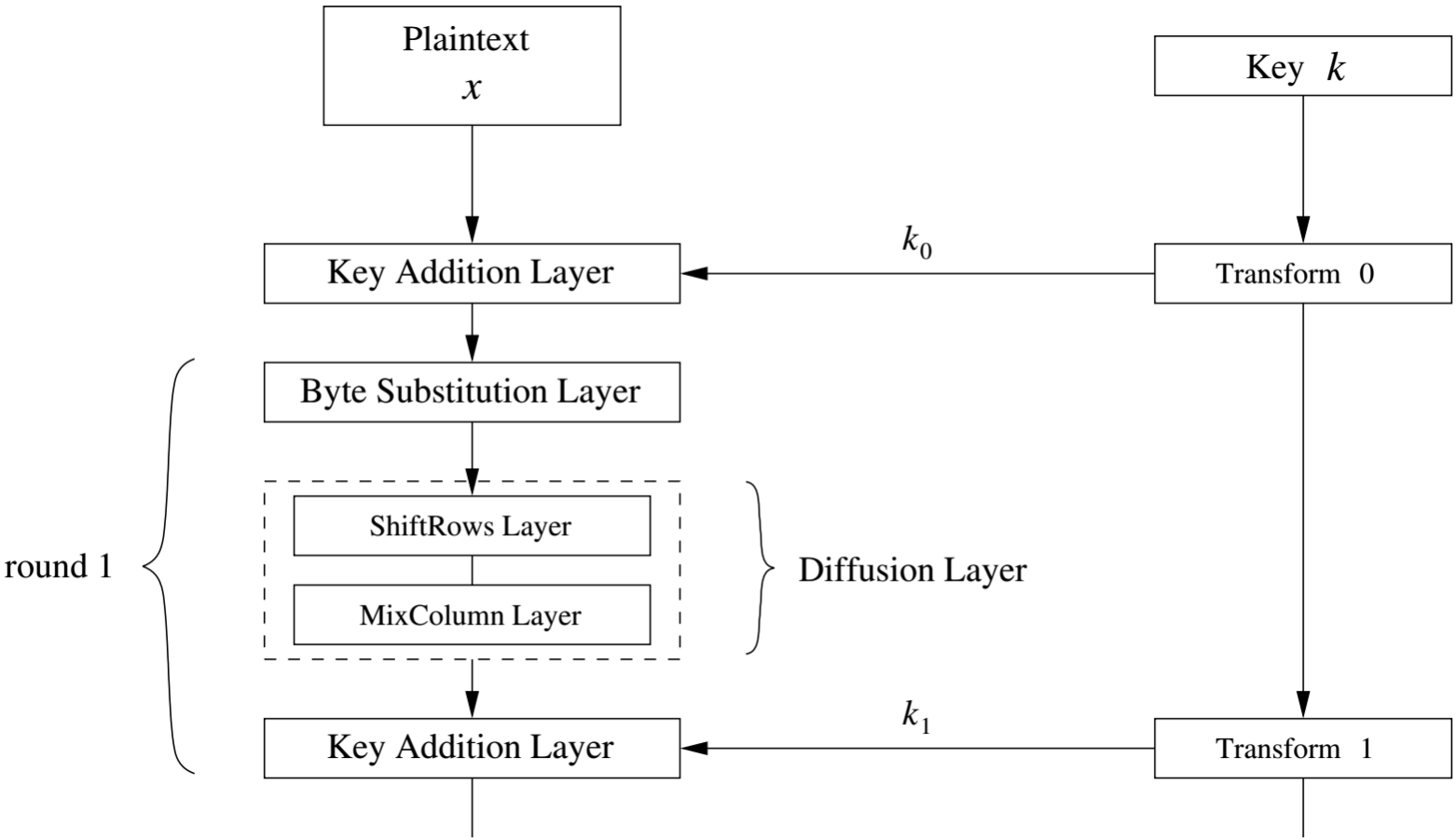
1. **SubBytes**
   Nonlinear byte substitution using the AES S-Box.
2. **ShiftRows**
   Permutes bytes by cyclically shifting the state rows.
3. **MixColumns**
   Mixes the bytes within each column using $\mathrm{GF}(2^8)$.
4. **AddRoundKey**
   Combines the state with the round key using XOR.

$$\textbf{128-bit state} \implies \textbf{16 bytes} \quad (16 \times 8 = 128)$$

Each transformation operates on the same 4×4 byte state.

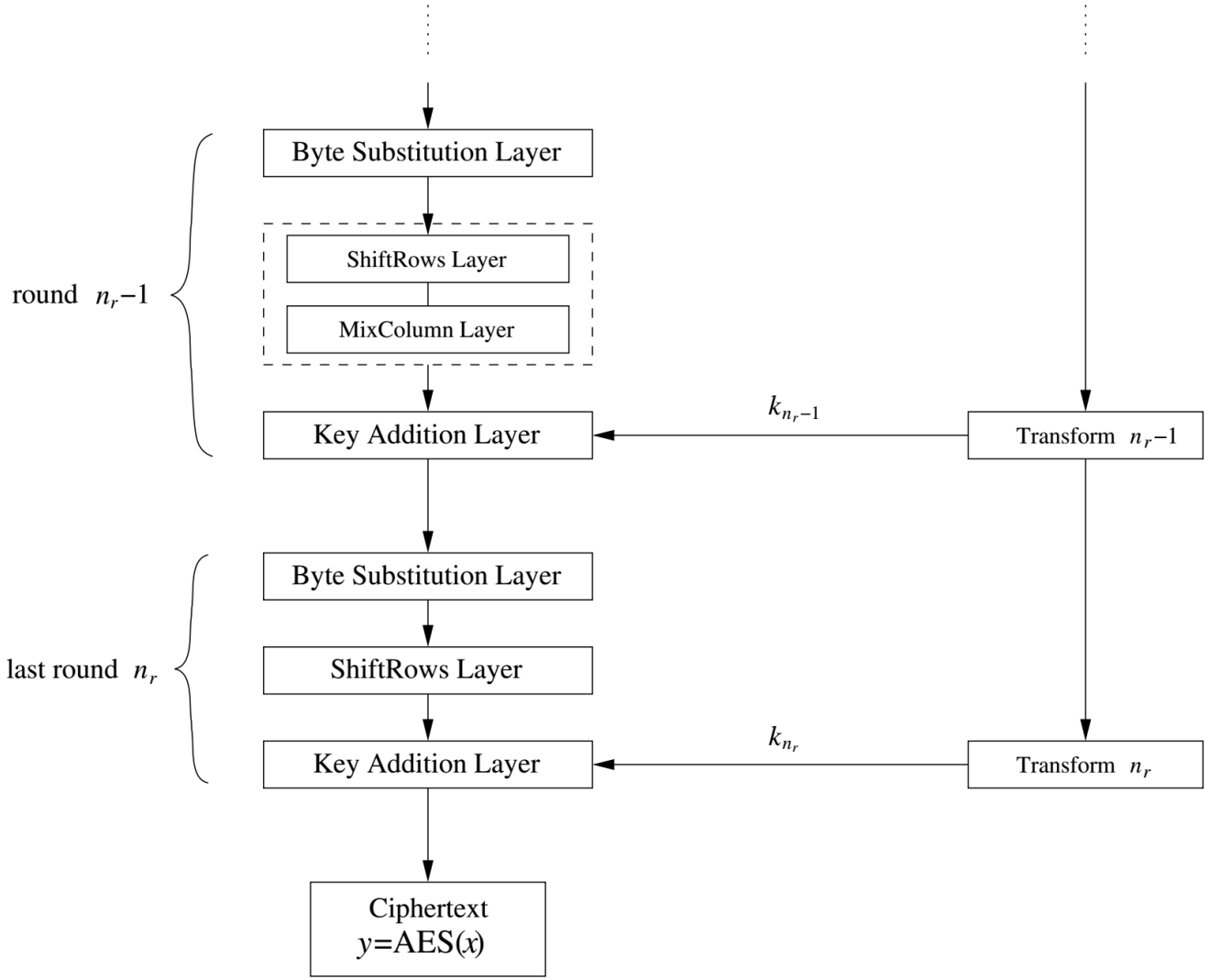
SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Structure of AES



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES encryption block diagram



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Introduction to AES

AES Round Transformations

1. SubBytes **Confusion**

Each byte of the state is transformed independently using the **AES S-Box**, providing a nonlinear substitution.

---

2. ShiftRows **Diffusion**

The rows of the state are cyclically shifted, changing the position of the bytes.

3. MixColumns

Each column is transformed using matrix multiplication in $\mathrm{GF}(2^8)$, spreading the influence of each byte.

---

4. AddRoundKey **Key Addition**

The 128-bit state is combined with a **128-bit round key** using bitwise XOR:

$$\text{State} \leftarrow \text{State} \oplus \text{RoundKey}$$

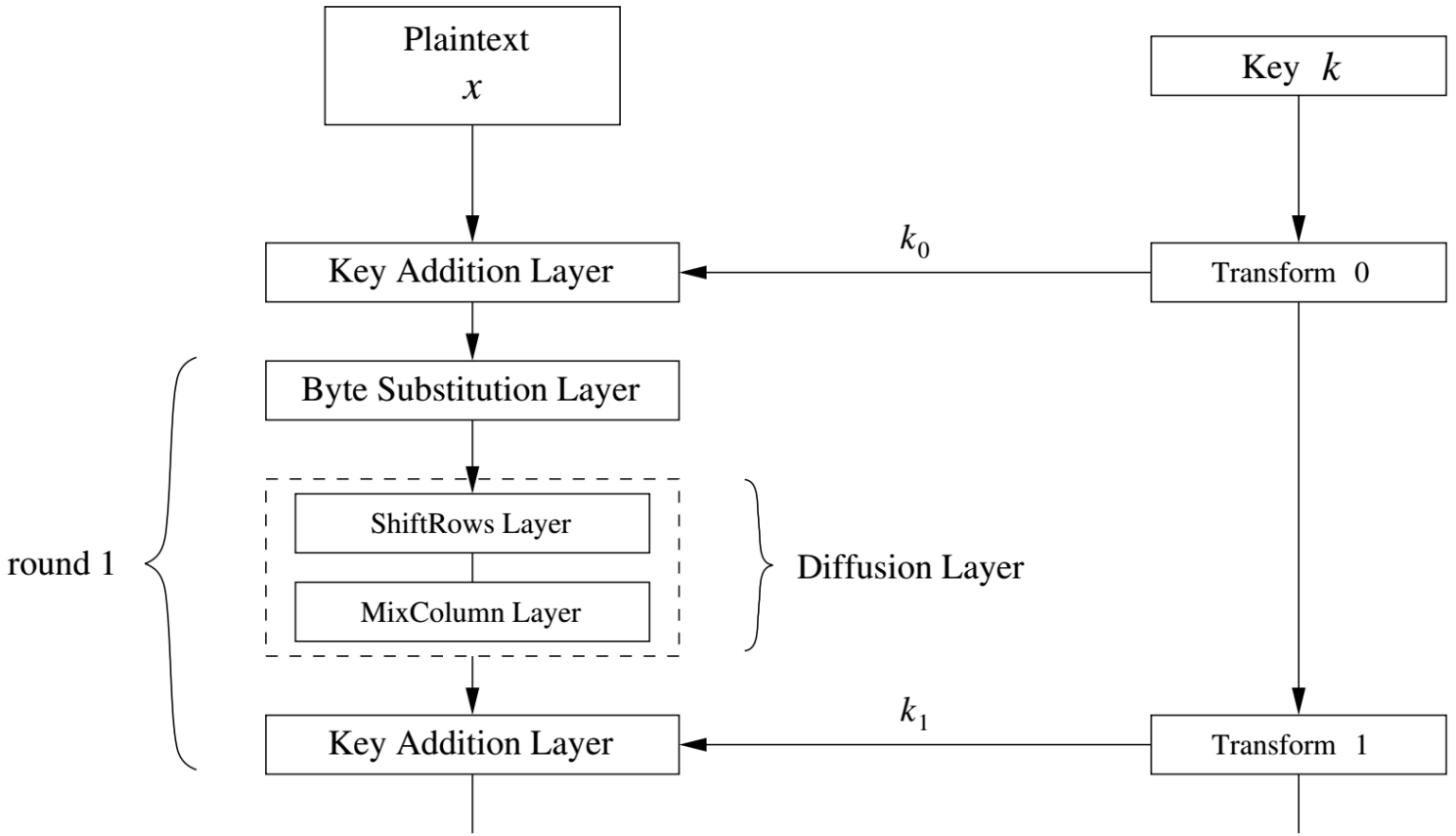
SubBytes provides nonlinearity, while **ShiftRows** and **MixColumns** provide diffusion.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Internal Structure of AES

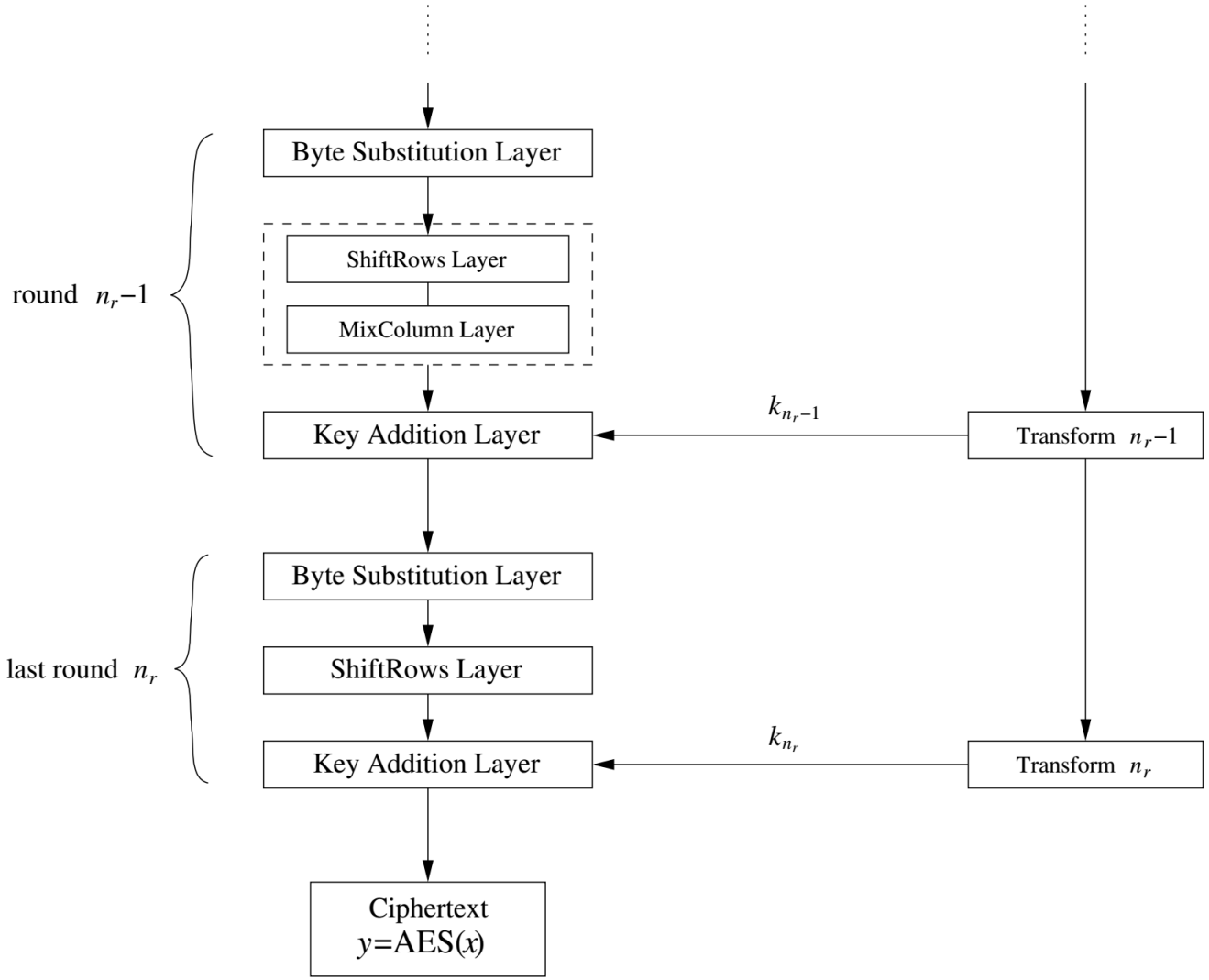
SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES encryption block diagram



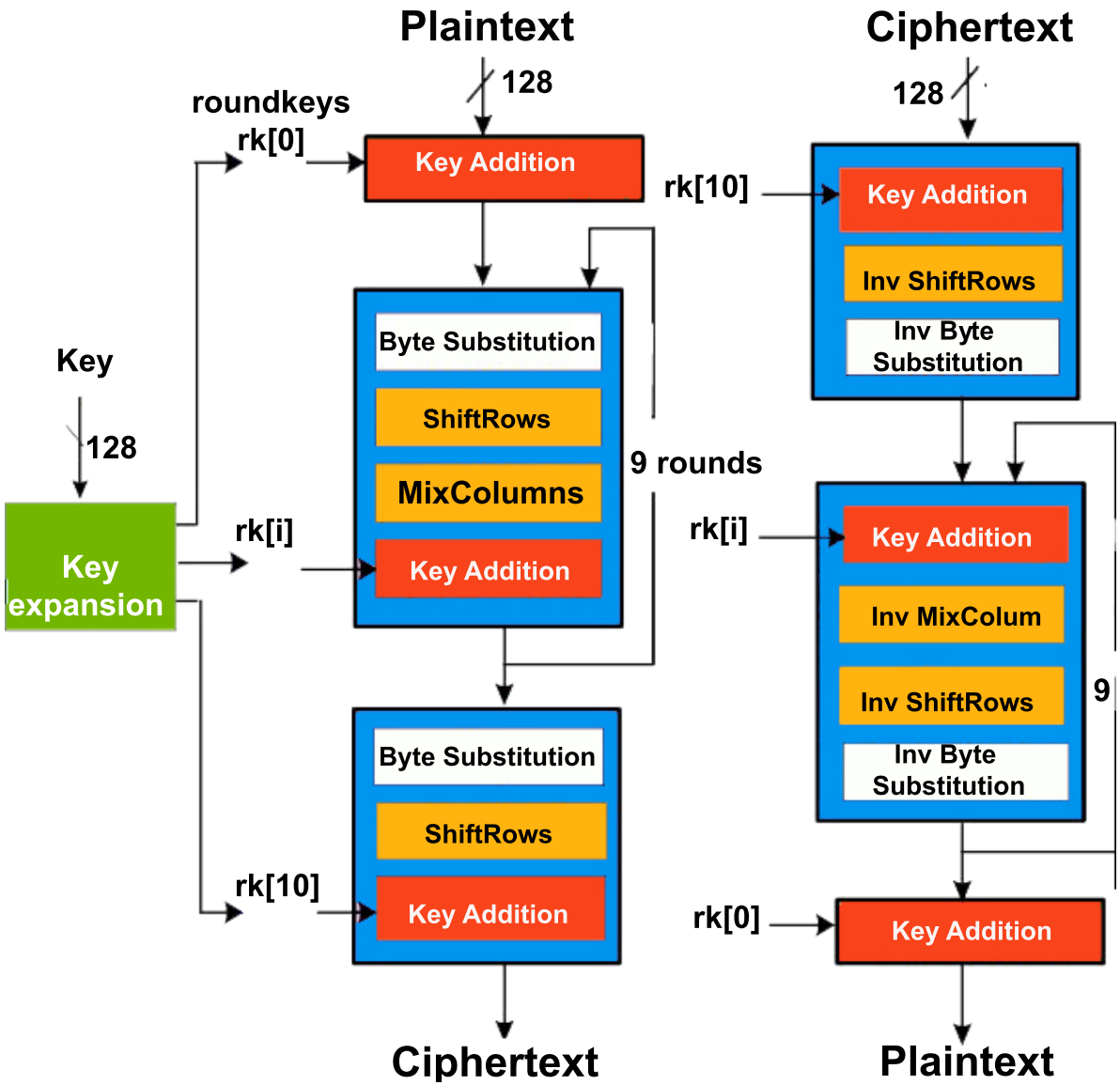
SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES encryption block diagram



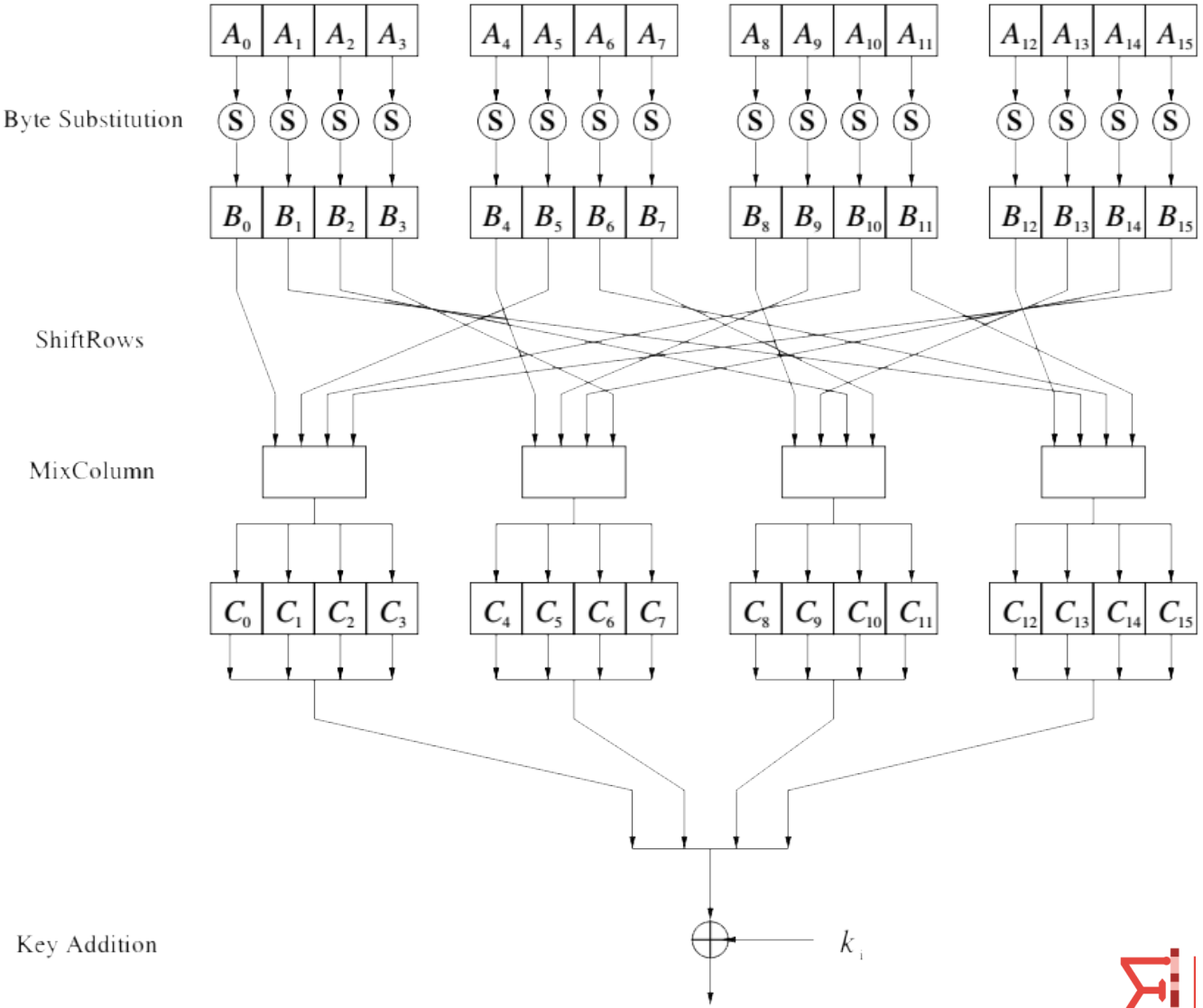
SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Internal Structure of AES



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Byte Substitution - ShiftRows - MixColumn - Key Addtion



AES round function for rounds $1, 2, \ldots, n_r - 1$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Internal Structure of AES

Data Flow Within an AES Round

1. **SubBytes**
   The 16-byte input state $A = (A_0, A_1, \ldots, A_{15})$ is transformed byte by byte using the same AES S-Box: $B_i = S(A_i)$
2. **ShiftRows**
   The 16 output bytes $B = (B_0, B_1, \ldots, B_{15})$ are permuted by cyclically shifting the rows of the state.
3. **MixColumns**
   Each 4-byte column is mixed through a linear transformation performed in $\mathrm{GF}(2^8)$.
4. **AddRoundKey**
   The intermediate 128-bit state isXORed with the **128-bit round key**

Internal Structure of AES

Byte Substitution Layer

- The AES state contains **16 bytes**:

$$A = (A_0, A_1, \ldots, A_{15}).$$

- During **SubBytes**, each byte is independently transformed using the AES S-Box:

$$\boxed{B_i = S(A_i)}$$

- The same **8-bit** $\rightarrow$ **8-bit S-Box** is applied to all 16 state bytes.
- This transformation introduces **nonlinearity** into the AES round.

$$A_i \longrightarrow \boxed{\text{AES S-Box}} \longrightarrow B_i$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

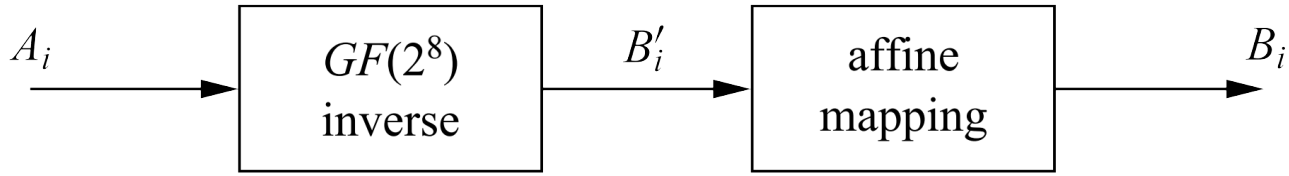
AES S-Box

AES S-Box: Substitution values in hexadecimal notation for input byte (xy)

x \ y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

How is the AES S-Box computed?



$\mathrm{GF}(2^8)$ Inverse

For each input byte A_i, AES first computes its **multiplicative inverse** in the finite field $\mathrm{GF}(2^8)$:

$$\boxed{B'_i = A_i^{-1}}$$

The field $\mathrm{GF}(2^8)$ is constructed using the fixed irreducible polynomial

$$\boxed{P(x) = x^8 + x^4 + x^3 + x + 1}$$

Thus, both A_i and B'_i represent 8-bit elements of the same finite field.

$$A_i \longrightarrow \textbf{Multiplicative Inverse in } \mathrm{GF}(2^8) \longrightarrow B'_i$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES S-Box

Multiplicative inverse table in $GF(2^8)$ for bytes xy used within the AES S-Box

X \ Y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	00	01	8D	F6	CB	52	7B	D1	E8	4F	29	C0	B0	E1	E5	C7
1	74	B4	AA	4B	99	2B	60	5F	58	3F	FD	CC	FF	40	EE	B2
2	3A	6E	5A	F1	55	4D	A8	C9	C1	0A	98	15	30	44	A2	C2
3	2C	45	92	6C	F3	39	66	42	F2	35	20	6F	77	BB	59	19
4	1D	FE	37	67	2D	31	F5	69	A7	64	AB	13	54	25	E9	09
5	ED	5C	05	CA	4C	24	87	BF	18	3E	22	F0	51	EC	61	17
6	16	5E	AF	D3	49	A6	36	43	F4	47	91	DF	33	93	21	3B
7	79	B7	97	85	10	B5	BA	3C	B6	70	D0	06	A1	FA	81	82
8	83	7E	7F	80	96	73	BE	56	9B	9E	95	D9	F7	02	B9	A4
9	DE	6A	32	6D	D8	8A	84	72	2A	14	9F	88	F9	DC	89	9A
A	FB	7C	2E	C3	8F	B8	65	48	26	C8	12	4A	CE	E7	D2	62
B	0C	E0	1F	EF	11	75	78	71	A5	8E	76	3D	BD	BC	86	57
C	0B	28	2F	A3	DA	D4	E4	0F	A9	27	53	04	1B	FC	AC	E6
D	7A	07	AE	63	C5	DB	E2	EA	94	8B	C4	D5	9D	F8	90	6B
E	B1	0D	D6	EB	C6	0E	CF	AD	08	4E	D7	E3	5D	50	1E	B3
F	5B	23	38	34	68	46	03	8C	DD	9C	7D	A0	CD	1A	41	1C

Byte Substitution layer

- Using inversion in GF(2^8) provides a high degree of nonlinearity, it provides protection against analytical attacks.

How is the AES S-Box computed?

Affine Mapping

After computing the multiplicative inverse

$$B_i' = A_i^{-1},$$

the 8 bits of B_i' are transformed using a **fixed binary matrix** and a **constant vector**.

$$\boxed{\mathbf{B}_i = \mathbf{M}\mathbf{B}_i' \oplus \mathbf{c}}$$

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} b_0' \\ b_1' \\ b_2' \\ b_3' \\ b_4' \\ b_5' \\ b_6' \\ b_7' \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

Key ideas

- All operations are performed in $\mathrm{GF}(2)$.
- Addition corresponds to **XOR**.
- **M** is a fixed 8×8 binary matrix.
- The constant vector corresponds to

$$\mathbf{c} = 63_{16}.$$

$$B_i' \longrightarrow \textbf{Affine Mapping} \longrightarrow B_i = S(A_i)$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES S-Box

Affine Transformation

After computing the multiplicative inverse

$$B_i' = A_i^{-1} \qquad \text{in } \mathrm{GF}(2^8),$$

the intermediate byte B_i' is transformed using a **fixed binary matrix** and a **constant 8-bit vector**.

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{pmatrix} \equiv \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} b_0' \\ b_1' \\ b_2' \\ b_3' \\ b_4' \\ b_5' \\ b_6' \\ b_7' \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \mod 2.$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Example: $A_i = (C2)_{16}$

Consider the S-Box input

$$A_i = (C2)_{16} = (11000010)_2.$$

1 Multiplicative inverse

In $\mathrm{GF}(2^8)$,

$$B_i' = A_i^{-1} = (2F)_{16} = (00101111)_2.$$

2 Affine transformation

The bit vector B_i' is multiplied by the fixed 8×8 binary matrix and XORed with $(63)_{16}$:

$$\mathbf{B}_i = \mathbf{M}\mathbf{B}_i' \oplus \mathbf{c}.$$

This produces

$$B_i = (25)_{16} = (00100101)_2.$$

3 S-Box output

Therefore,

$$\boxed{S(C2) = 25}.$$

Bit convention: b_0' is the least significant bit (LSB).

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES S-Box

Why is the AES S-Box Nonlinear?

1. **Multiplicative inverse**

$$A_i \longrightarrow A_i^{-1}$$

introduces the main **nonlinear component** of the AES S-Box.

2. **Affine mapping**

$$B_i' \longrightarrow \mathbf{MB}_i' \oplus \mathbf{c}$$

modifies the algebraic structure of the result using a fixed binary transformation.

Inverse $+$ **Affine Mapping** $=$ **AES S-Box**

This construction provides strong nonlinearity and contributes to AES resistance against **linear and differential cryptanalysis**.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Exercise 1 – Affine Transformation

Consider the input byte

$$A_i = (53)_{16}.$$

Its multiplicative inverse in $\mathrm{GF}(2^8)$ is given:

$$B_i' = (CA)_{16} = (11001010)_2.$$

Using the AES affine transformation

$$\mathbf{B}_i = \mathbf{M}\mathbf{B}_i' \oplus \mathbf{c},$$

solve the following:

1. Write B_i' as the corresponding 8-bit vector.
2. Multiply the vector by the AES constant binary matrix $\mathbf{M}$.
3. XOR the result with the constant vector

$$\mathbf{c} = (63)_{16}.$$

4. Express the final result in binary and hexadecimal.
5. Determine the S-Box value:

$$\boxed{S(53) = ?}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Exercise 2 – Complete S-Box Computation

Consider the input byte

$$A_i = (57)_{16}.$$

Perform the complete AES S-Box construction.

1. Convert A_i to binary and polynomial representation.
2. Compute the multiplicative inverse

   $$B_i' = A_i^{-1}$$

   in $\mathrm{GF}(2^8)$ using

   $$P(x) = x^8 + x^4 + x^3 + x + 1.$$

3. Apply the AES affine transformation:

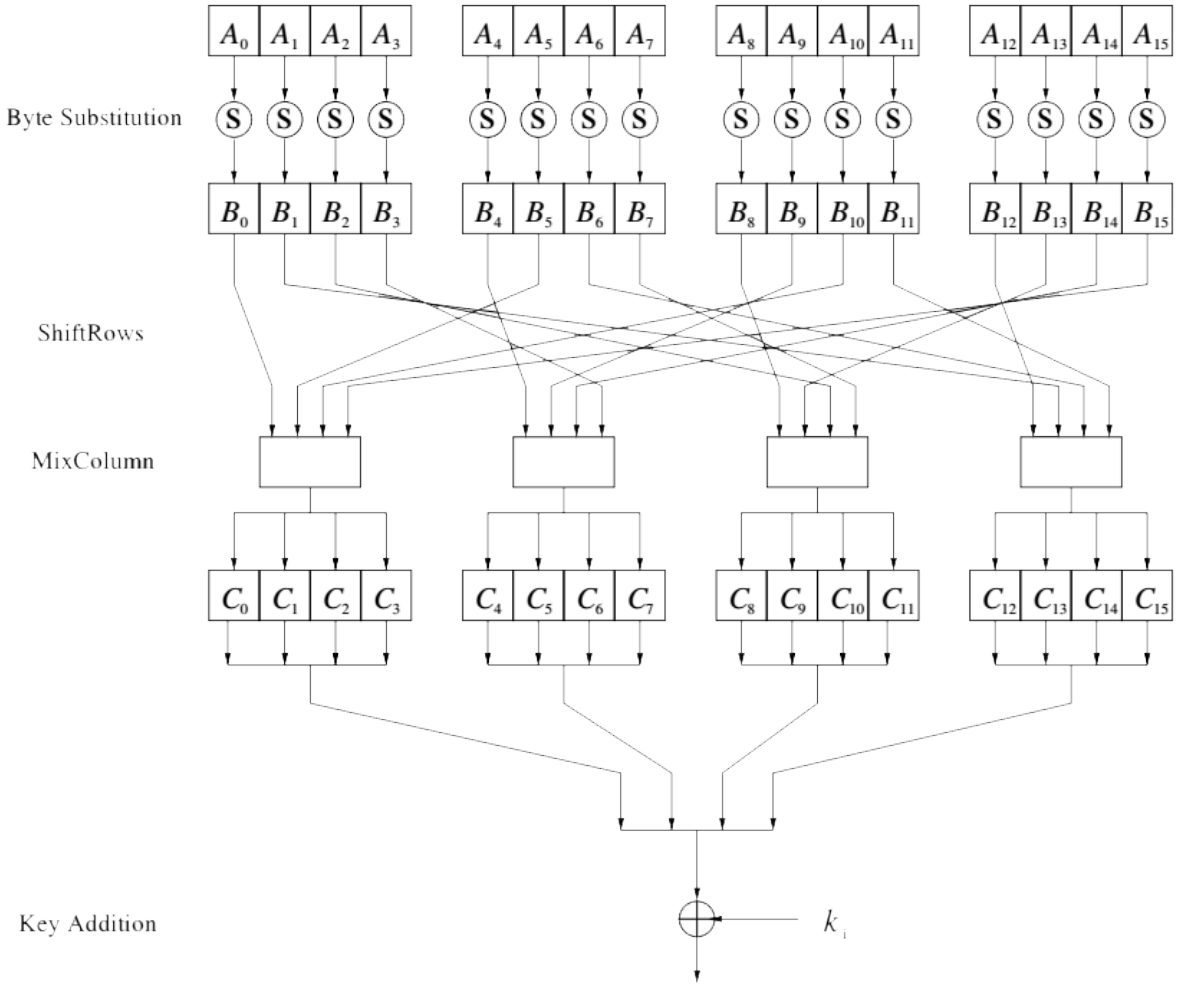
   $$\mathbf{B}_i = \mathbf{M}\mathbf{B}_i' \oplus \mathbf{c}.$$

4. Express the final S-Box output in binary and hexadecimal.
5. Determine:

$$\boxed{S(57) = ?}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Byte Substitution - ShiftRows - MixColumn - Key Addtion



AES round function for rounds $1, 2, \ldots, n_r - 1$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Diffusion in AES

Diffusion Transformations

AES achieves diffusion through two transformations:

ShiftRows

Permutes the bytes across the columns of the state.

$\Longrightarrow$

MixColumns

Mixes the four bytes within each column.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

ShiftRows Transformation

How does ShiftRows work?

The 16-byte AES state is arranged as a 4×4 byte matrix. During **ShiftRows**, each row is cyclically shifted to the left:

B_0	B_4	B_8	B_{12}
B_1	B_5	B_9	B_{13}
B_2	B_6	B_{10}	B_{14}
B_3	B_7	B_{11}	B_{15}

B_0	B_4	B_8	B_{12}	no shift
B_5	B_9	B_{13}	B_1	$\longrightarrow$ three positions right shift
B_{10}	B_{14}	B_2	B_6	$\longrightarrow$ two positions right shift
B_{15}	B_3	B_7	B_{11}	$\longrightarrow$ one position right shift

- **Row 0:** no shift
- **Row 1:** shift by 1 byte
- **Row 2:** shift by 2 bytes
- **Row 3:** shift by 3 bytes

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

ShiftRows Layer

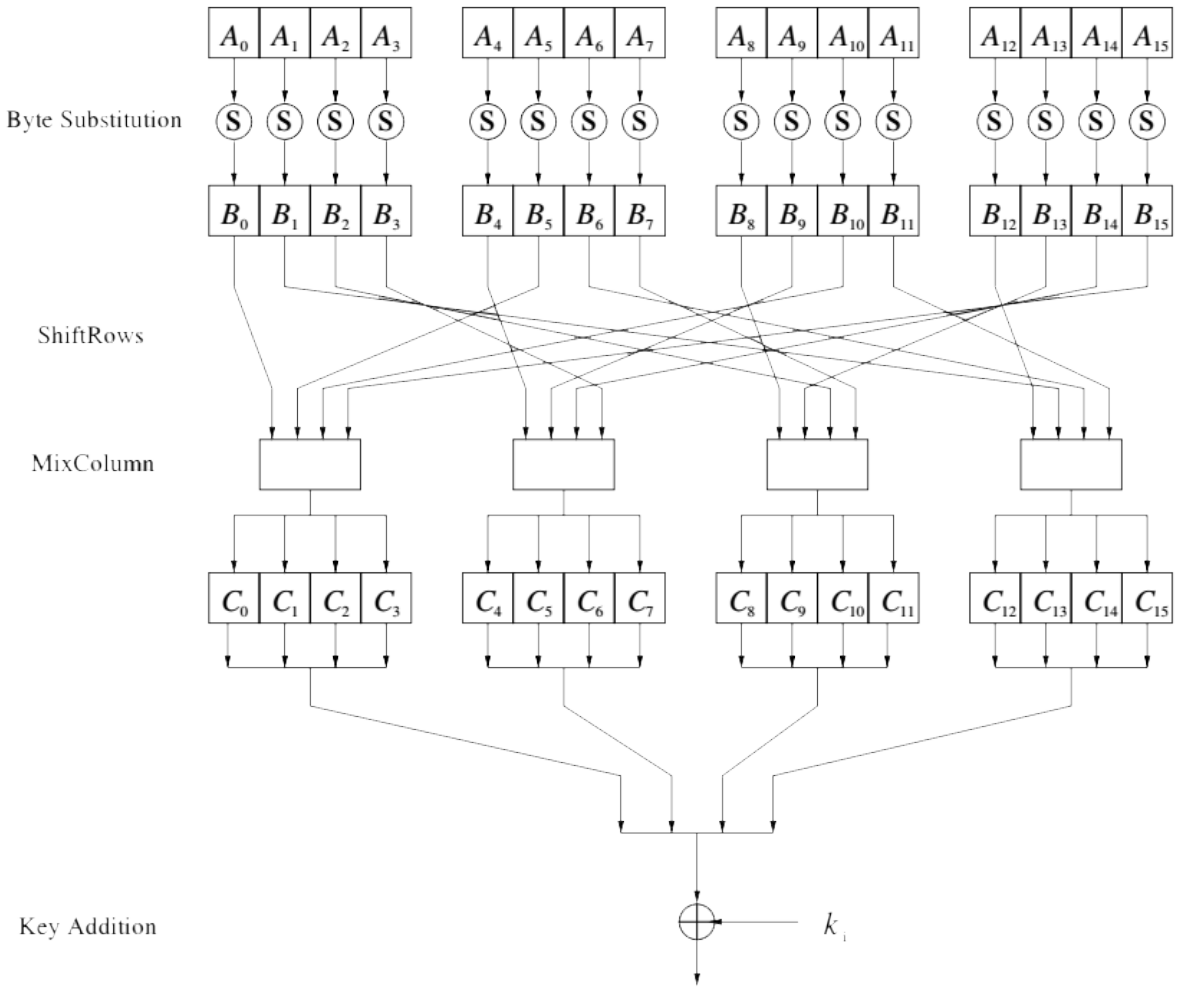
If the input of the ShiftRows sublayer is given as a state matrix $B = (B_0, B_1, ..., B_{15})$:

B_0	B_4	B_8	B_{12}
B_1	B_5	B_9	B_{13}
B_2	B_6	B_{10}	B_{14}
B_3	B_7	B_{11}	B_{15}

B_0	B_4	B_8	B_{12}	no shift
B_5	B_9	B_{13}	B_1	$\longrightarrow$ three positions right shift
B_{10}	B_{14}	B_2	B_6	$\longrightarrow$ two positions right shift
B_{15}	B_3	B_7	B_{11}	$\longrightarrow$ one position right shift

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Byte Substitution - ShiftRows - MixColumn - Key Addtion



AES round function for rounds $1, 2, \ldots, n_r - 1$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

MixColumns Transformation

How does MixColumns work?

After **ShiftRows**, AES applies a linear transformation **independently to each column** of the state.

B_0	B_4	B_8	B_{12}
B_5	B_9	B_{13}	B_1
B_{10}	B_{14}	B_2	B_6
B_{15}	B_3	B_7	B_{11}

Each column contains four bytes:

$$\begin{bmatrix} B_0 \\ B_1 \\ B_2 \\ B_3 \end{bmatrix} \longrightarrow \begin{bmatrix} C_0 \\ C_1 \\ C_2 \\ C_3 \end{bmatrix}$$

The transformation is performed using a fixed 4×4 matrix over

$$\mathrm{GF}(2^8).$$

$$\boxed{C = \mathrm{MixColumns}(B)}$$

where B is the state after the **ShiftRows** transformation.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

MixColumns Transformation

Matrix Operation in MixColumns

Each 4-byte column of the AES state is treated as a vector and multiplied by a fixed 4×4 matrix.

$$\begin{pmatrix} C_0 \\ C_1 \\ C_2 \\ C_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} B_0 \\ B_5 \\ B_{10} \\ B_{15} \end{pmatrix}$$

All matrix coefficients are fixed AES constants.

Multiplication and addition are performed in $\mathrm{GF}(2^8)$.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

MixColumn Layer

$$\begin{pmatrix} C_0 \\ C_1 \\ C_2 \\ C_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} B_0 \\ B_5 \\ B_{10} \\ B_{15} \end{pmatrix}$$

$$\begin{pmatrix} C_4 \\ C_5 \\ C_6 \\ C_7 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} B_4 \\ B_9 \\ B_{14} \\ B_3 \end{pmatrix}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

MixColumn Layer

$$\begin{pmatrix} C_8 \\ C_9 \\ C_{10} \\ C_{11} \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} B_8 \\ B_{13} \\ B_2 \\ B_7 \end{pmatrix}$$

$$\begin{pmatrix} C_{12} \\ C_{13} \\ C_{14} \\ C_{15} \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} B_{12} \\ B_1 \\ B_6 \\ B_{11} \end{pmatrix}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

MixColumns Transformation

Constants Used in MixColumns

The MixColumns matrix contains only three constants:

$$01, \qquad 02, \qquad 03.$$

These hexadecimal values represent elements of $\mathrm{GF}(2^8)$ and therefore correspond to binary polynomials.

Hex	Binary	Polynomial
01	00000001	1
02	00000010	x
03	00000011	$x+1$

Therefore, multiplication by the MixColumns constants means

$$01 \cdot B = B,$$

$$02 \cdot B = x \cdot B,$$

$$03 \cdot B = (x+1) \cdot B.$$

AND
L SCIENCES

Example: Multiplication in $\mathrm{GF}(2^8)$

Consider one state column in which all four bytes are

$$B = \begin{bmatrix} 25 \\ 25 \\ 25 \\ 25 \end{bmatrix}_{16}.$$

Since

$$(25)_{16} = (00100101)_2,$$

its polynomial representation is

$$(25)_{16} \longleftrightarrow x^5 + x^2 + 1.$$

Only the products by 02 and 03 must be computed:

$$\begin{aligned} 02 \cdot 25 &= x(x^5 + x^2 + 1) \\ &= x^6 + x^3 + x, \end{aligned}$$

and

$$\begin{aligned} 03 \cdot 25 &= (x+1)(x^5 + x^2 + 1) \\ &= (x^6 + x^3 + x) + (x^5 + x^2 + 1) \\ &= x^6 + x^5 + x^3 + x^2 + x + 1. \end{aligned}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Computing the Output Column

From the previous calculation,

$$02 \cdot 25 = (4A)_{16}, \qquad 03 \cdot 25 = (6F)_{16}.$$

Since both polynomial products have degree smaller than 8, no reduction modulo

$$P(x) = x^8 + x^4 + x^3 + x + 1$$

is required.

The first output byte is

$$C_0 = (02 \cdot 25) \oplus (03 \cdot 25) \oplus (01 \cdot 25) \oplus (01 \cdot 25).$$

Therefore,

$$\begin{aligned} C_0 &= 4A \oplus 6F \oplus 25 \oplus 25 \\ &= 25. \end{aligned}$$

Because all four input bytes are identical,

$$\boxed{C = \begin{bmatrix} 25 \\ 25 \\ 25 \\ 25 \end{bmatrix}_{16}}$$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Diffusion Layer

MixColumn Layer

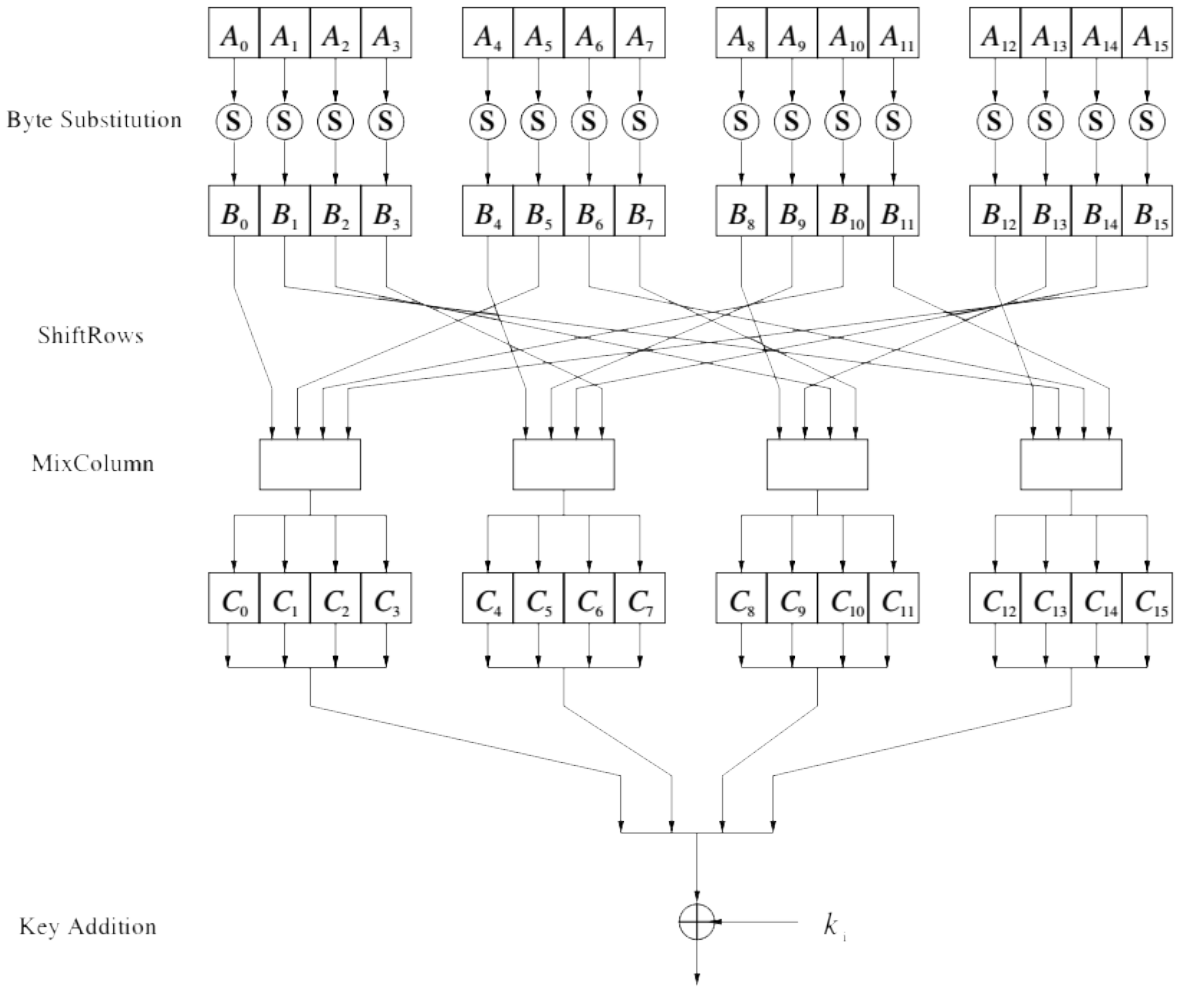
Since both intermediate values have a degree smaller than 8, no modular reduction with $P(x)$ is necessary. The output bytes of C result from the following addition in GF(2^8):

$$\begin{array}{rcccccccccccc}
01 \cdot 25 = & & & & x^5 + & & & & x^2 + & & 1 \\
01 \cdot 25 = & & & & x^5 + & & & & x^2 + & & 1 \\
02 \cdot 25 = & x^6 + & & & & x^3 + & & & & x & \\
03 \cdot 25 = & x^6 + & x^5 + & & x^3 + & & x^2 + & & x + & & 1 \\
\hline
C_i = & & & & x^5 + & & & & x^2 + & & 1,
\end{array}$$

This leads to the output state $C = (25, 25, 25, 25, 25, 25, 25, 25)$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Byte Substitution - ShiftRows - MixColumn - Key Addtion



AES round function for rounds $1, 2, \ldots, n_r - 1$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AddRoundKey Transformation

How does AddRoundKey work?

The AddRoundKey transformation combines two 128-bit values:

$$\textbf{State} \oplus \textbf{Round Key} \longrightarrow \textbf{New State}$$

- The AES state contains **16 bytes = 128 bits**.
- The round key K_i, generated by the **key schedule**, also contains **16 bytes = 128 bits**.
- The state and the round key are combined using a **bitwise XOR** operation:

$$\boxed{\text{State}_{\text{out}} = \text{State}_{\text{in}} \oplus K_i}$$

AddRoundKey is the transformation that introduces the secret key material into each AES round.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AddRoundKey Transformation

Example of AddRoundKey

Consider one byte of the AES state and the corresponding byte of the round key:

$$\text{State byte} = (3C)_{16}$$

$$\text{Round-key byte} = (A5)_{16}$$

Convert both values to binary:

$$(3C)_{16} = (00111100)_2$$

$$(A5)_{16} = (10100101)_2$$

The AddRoundKey operation performs a bitwise XOR:

$$\begin{array}{r} 00111100 \\ \oplus\ 10100101 \\ \hline 10011001 \end{array}$$

Therefore,

$$\boxed{(3C)_{16} \oplus (A5)_{16} = (99)_{16}}$$

AND
L SCIENCES

AES Key Schedule

How many Round Keys are generated?

The AES key schedule expands the original secret key into a set of **128-bit round keys**.

The number of round keys is

$$\boxed{N_{\text{keys}} = N_r + 1}$$

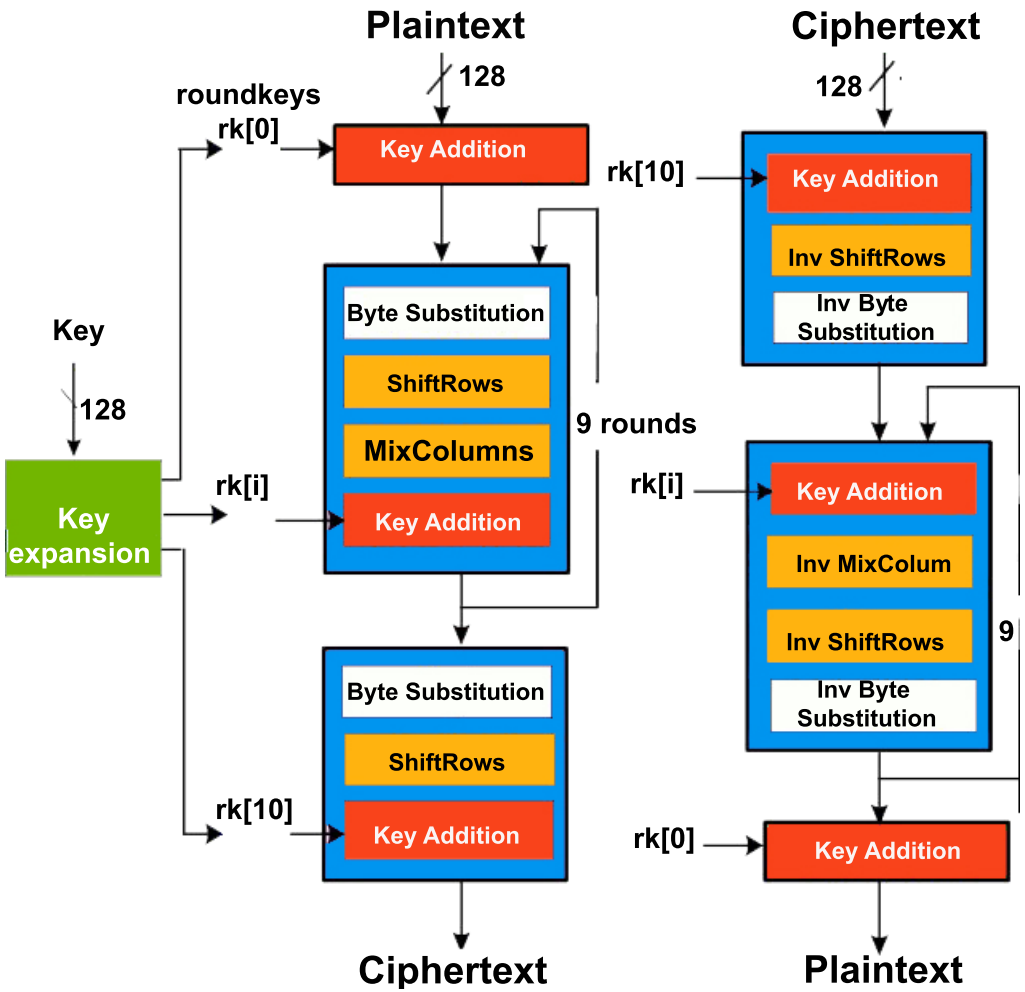
because AES uses one key before the first round and one key in each round.

Key Size	**Rounds** N_r	**Round Keys**
128 bits	10	11
192 bits	12	13
256 bits	14	15

Each round key has a fixed size of $\boxed{128 \text{ bits} = 16 \text{ bytes}}$ independently of the original AES key size.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Key Schedule



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES Key Schedule

Word-Oriented Key Expansion

AES-128 starts with a **128-bit cipher key**, divided into four 32-bit words:

$$K = [W_0\ W_1\ W_2\ W_3]$$

- One AES word contains

$$\boxed{1 \text{ word} = 32 \text{ bits} = 4 \text{ bytes}}$$

- The key schedule expands the original 4 words into

$$\boxed{W_0, W_1, \ldots, W_{43}}$$

  giving a total of **44 words**.
- Every four consecutive words form one **128-bit round key**:

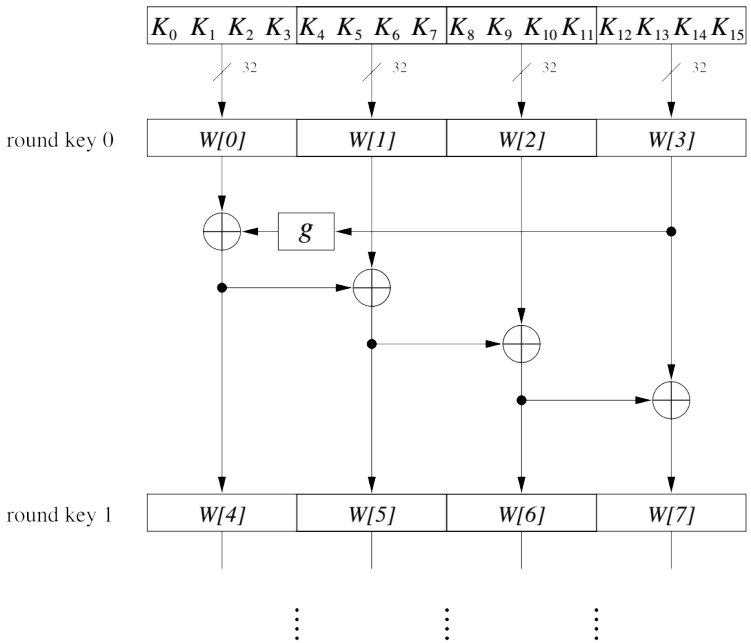
$$K_r = [W_{4r}, W_{4r+1}, W_{4r+2}, W_{4r+3}]\,.$$

AES-128: 44 words $\Rightarrow$ 11 round keys $\Rightarrow$ 176 bytes.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES-128 Key Schedule

Generating the Expanded Words



For AES-128, the expanded words are generated recursively.

Normally,

$$W_i = W_{i-4} \oplus W_{i-1}.$$

However, when

$$i \equiv 0 \pmod 4,$$

a special transformation is applied:

$$\boxed{W_i = W_{i-4} \oplus g(W_{i-1})}$$

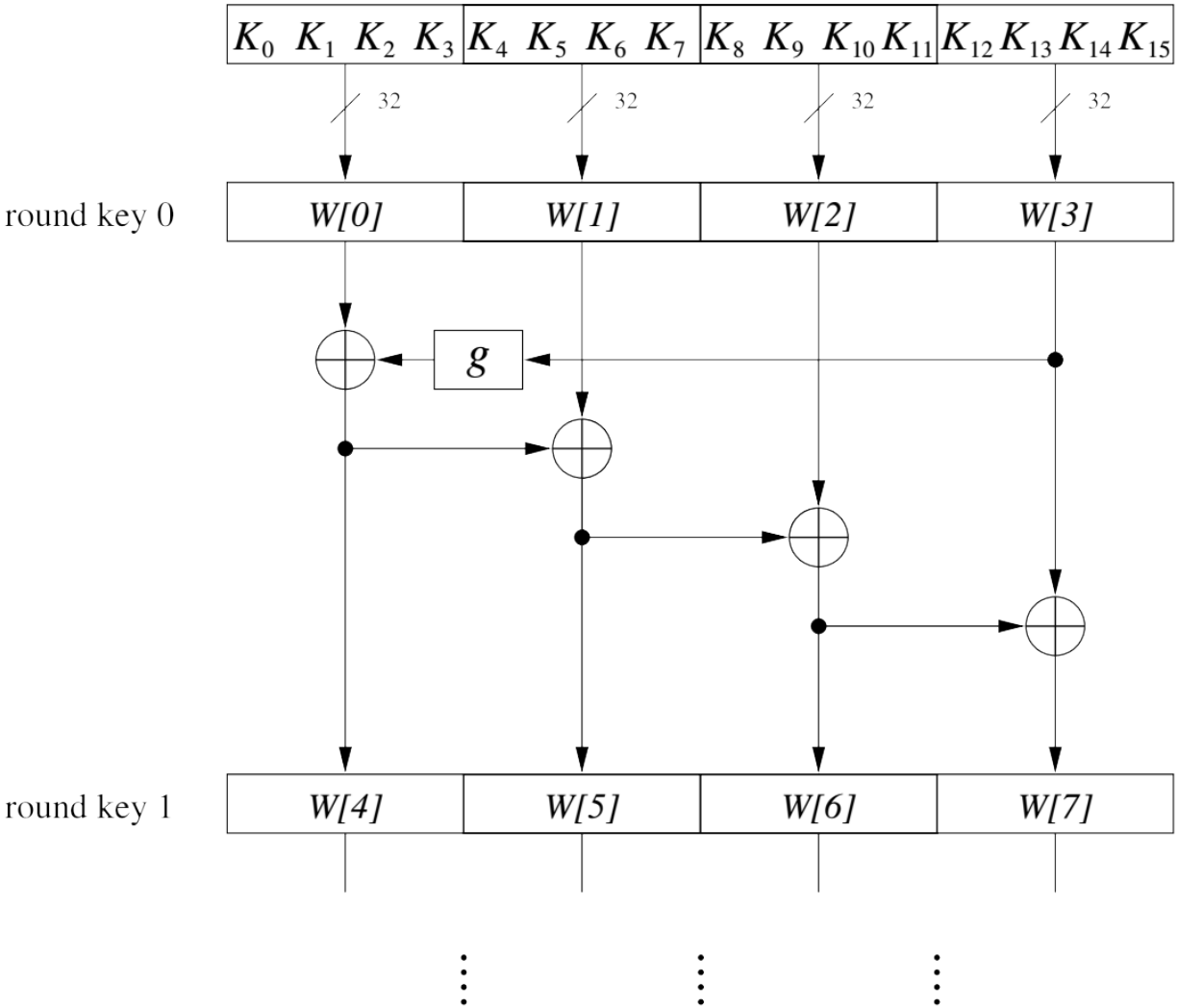
where

$$g(W) = \text{SubWord}\left(\text{RotWord}(W)\right) \oplus \text{Rcon}.$$

The function $g(\cdot)$ is applied once every four generated words.

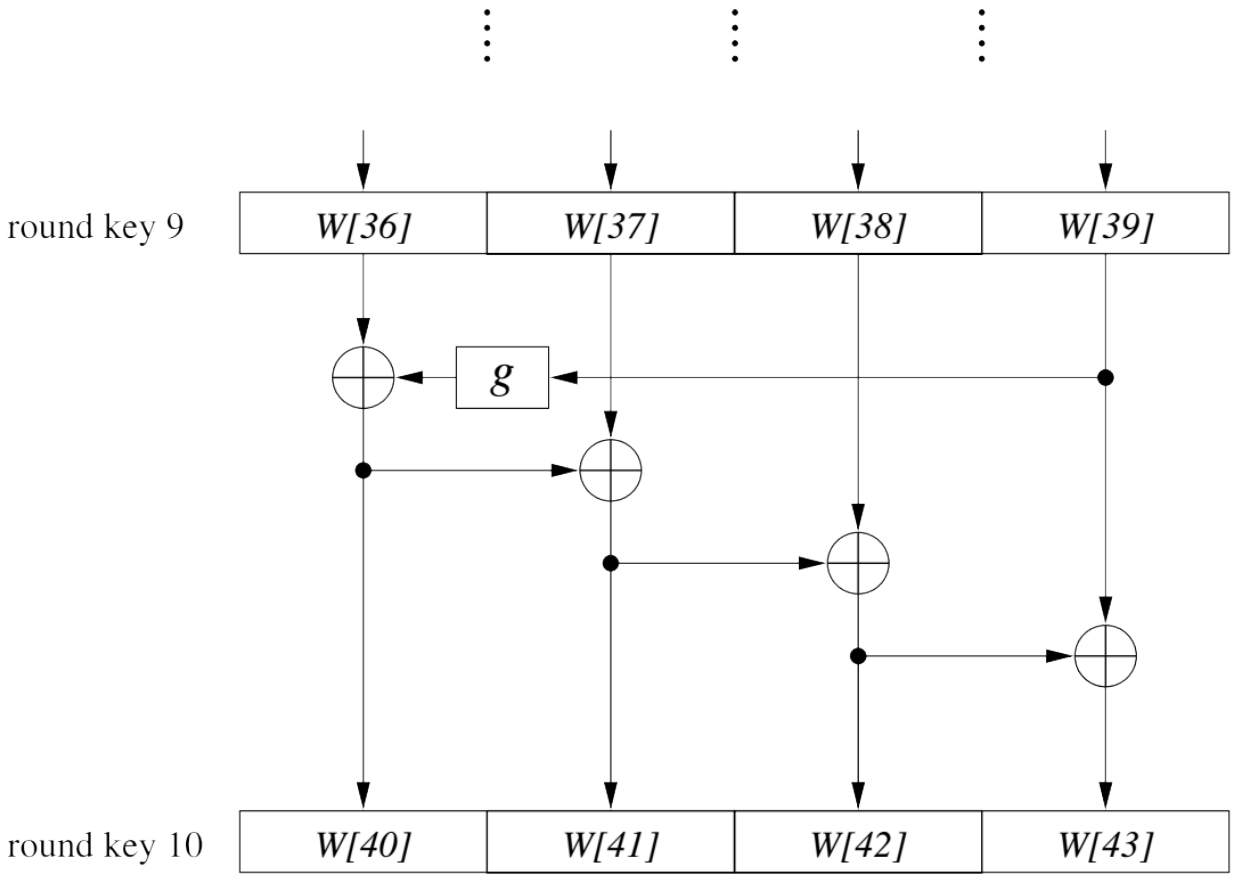
SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Key Schedule for 128-Bit Key AES



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Key Schedule for 128-Bit Key AES



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES-128 Key Schedule

Function $g(W)$

In AES-128, the function $g(W)$ is applied to one 32-bit word before generating every fourth expanded word.

$W \longrightarrow$ **RotWord** $\longrightarrow$ **SubWord** $\longrightarrow$ **RCon** $\longrightarrow$

1. **RotWord**
   Cyclically rotates the four bytes one position to the left:
   $$[a_0, a_1, a_2, a_3] \longrightarrow [a_1, a_2, a_3, a_0]$$
2. **SubWord**
   Applies the AES S-Box independently to each byte:
   $$[a_1, a_2, a_3, a_0] \longrightarrow [S(a_1), S(a_2), S(a_3), S(a_0)]$$
3. **Rcon**
   The result is XORed with the round constant:
   $$g(W) = \text{SubWord}(\text{RotWord}(W)) \oplus \text{RCon}_i$$

AND
L SCIENCES

AES-128 Key Schedule

Round Constants – Rcon

The function $g(W)$ uses a different **round constant** for each round.

The round constant is represented as a 32-bit word:

$$\boxed{\text{Rcon}_i = [RC_i,\ 00,\ 00,\ 00]}$$

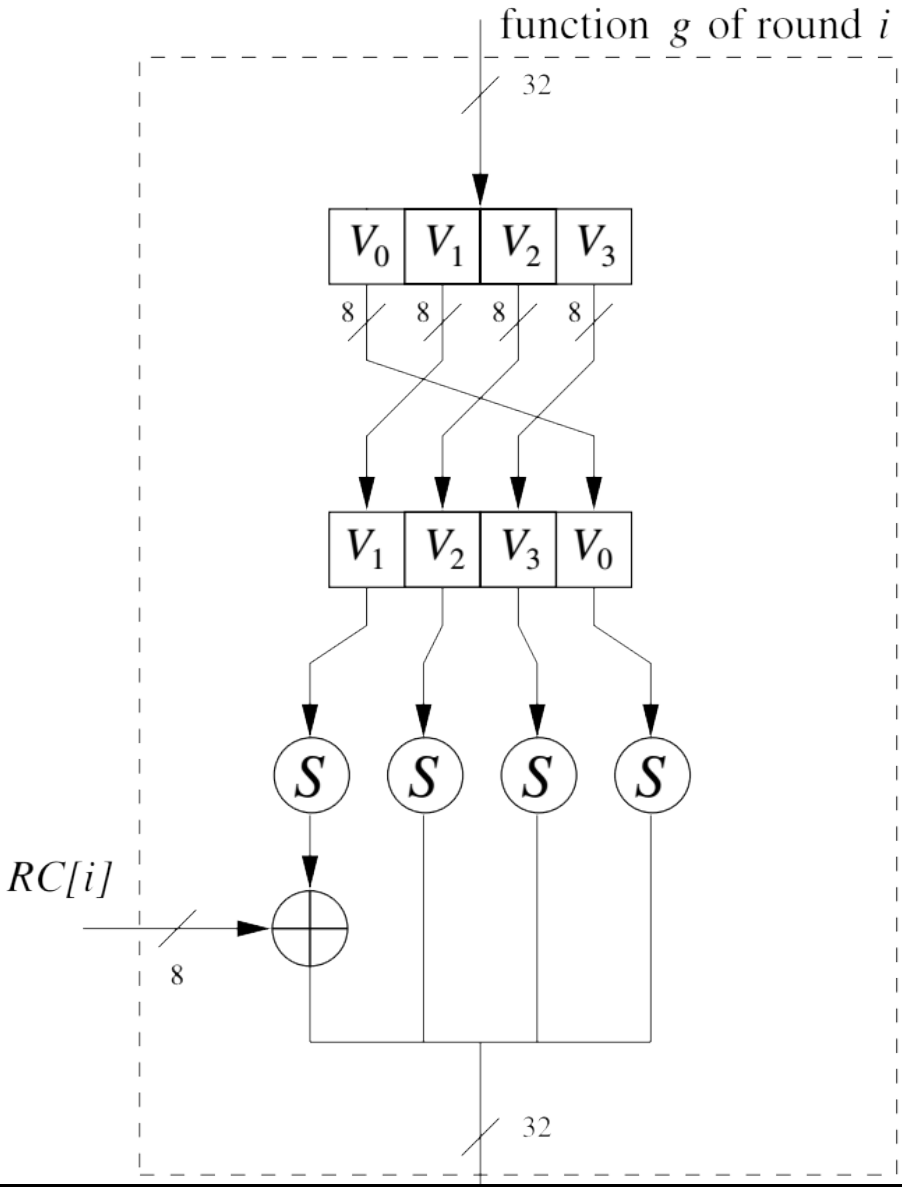
where RC_i is an 8-bit element of $\text{GF}(2^8)$.

For AES-128:

Round	RC_i	Hex
1	x^0	01
2	x^1	02
3	x^2	04
4	x^3	08
5	x^4	10
6	x^5	20
7	x^6	40
8	x^7	80
9	x^8	1B
10	x^9	36

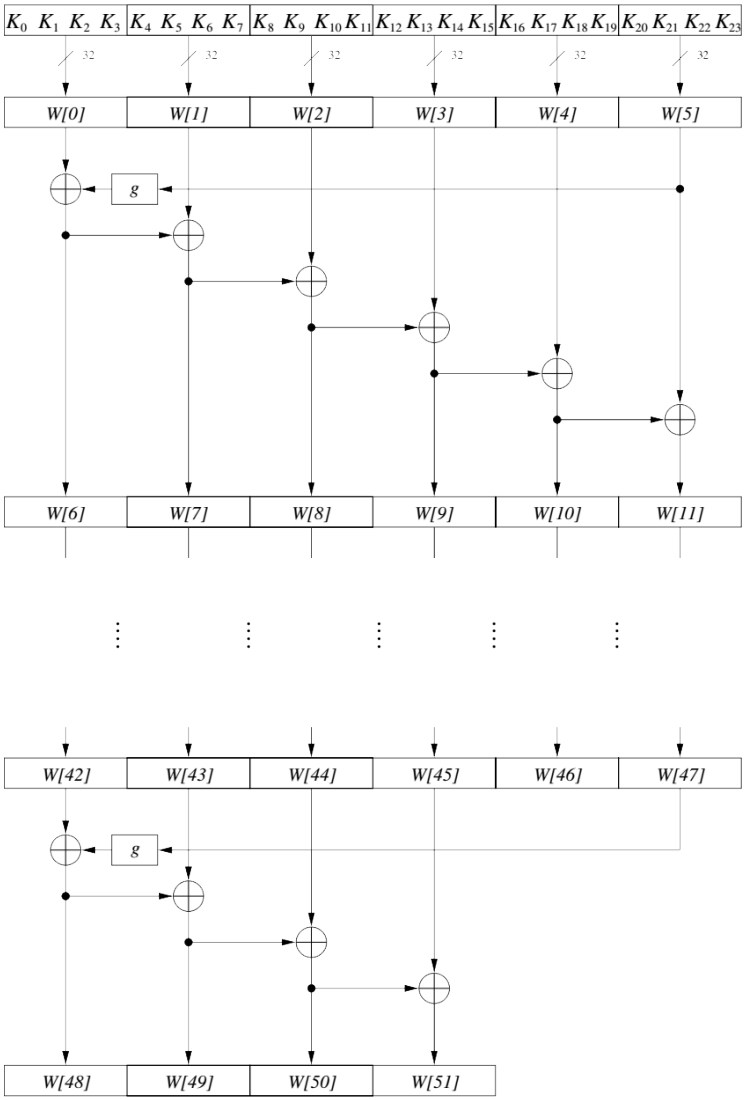
AND
L SCIENCES

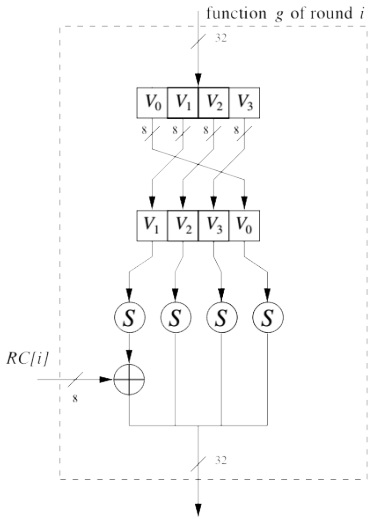
Key Schedule for 128-Bit Key AES



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

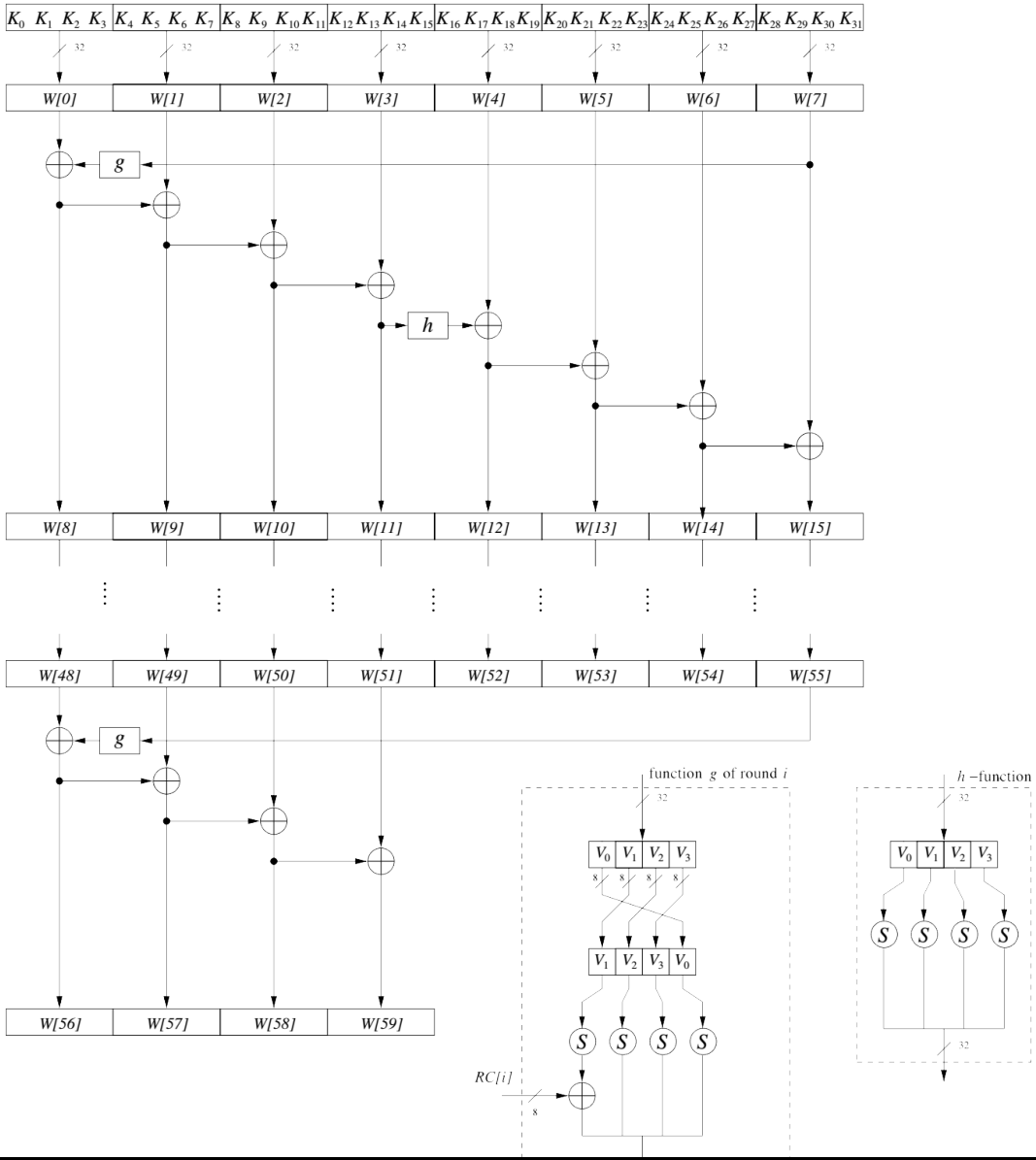
Key Schedule for 192-Bit Key AES





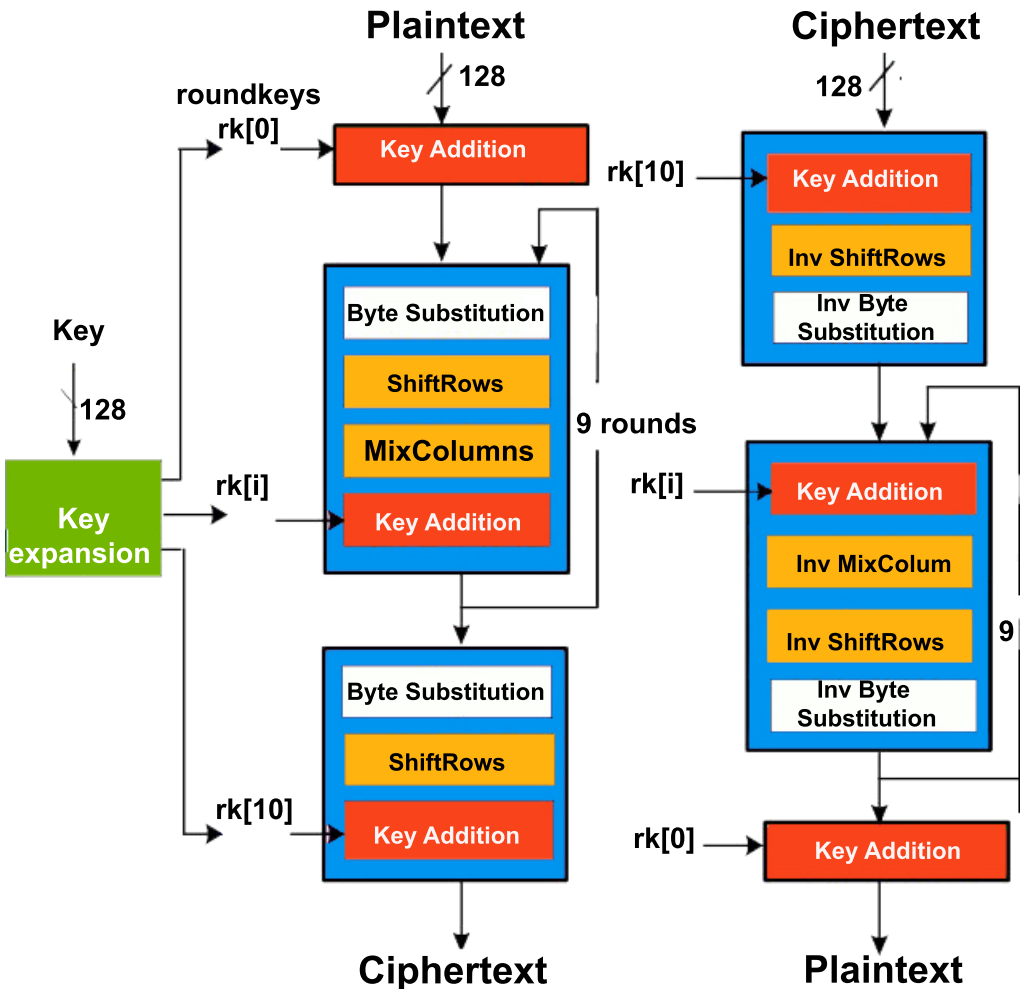
SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Key Schedule for 256-Bit Key AES



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES Decryption



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES Decryption

Inverse AES Transformations

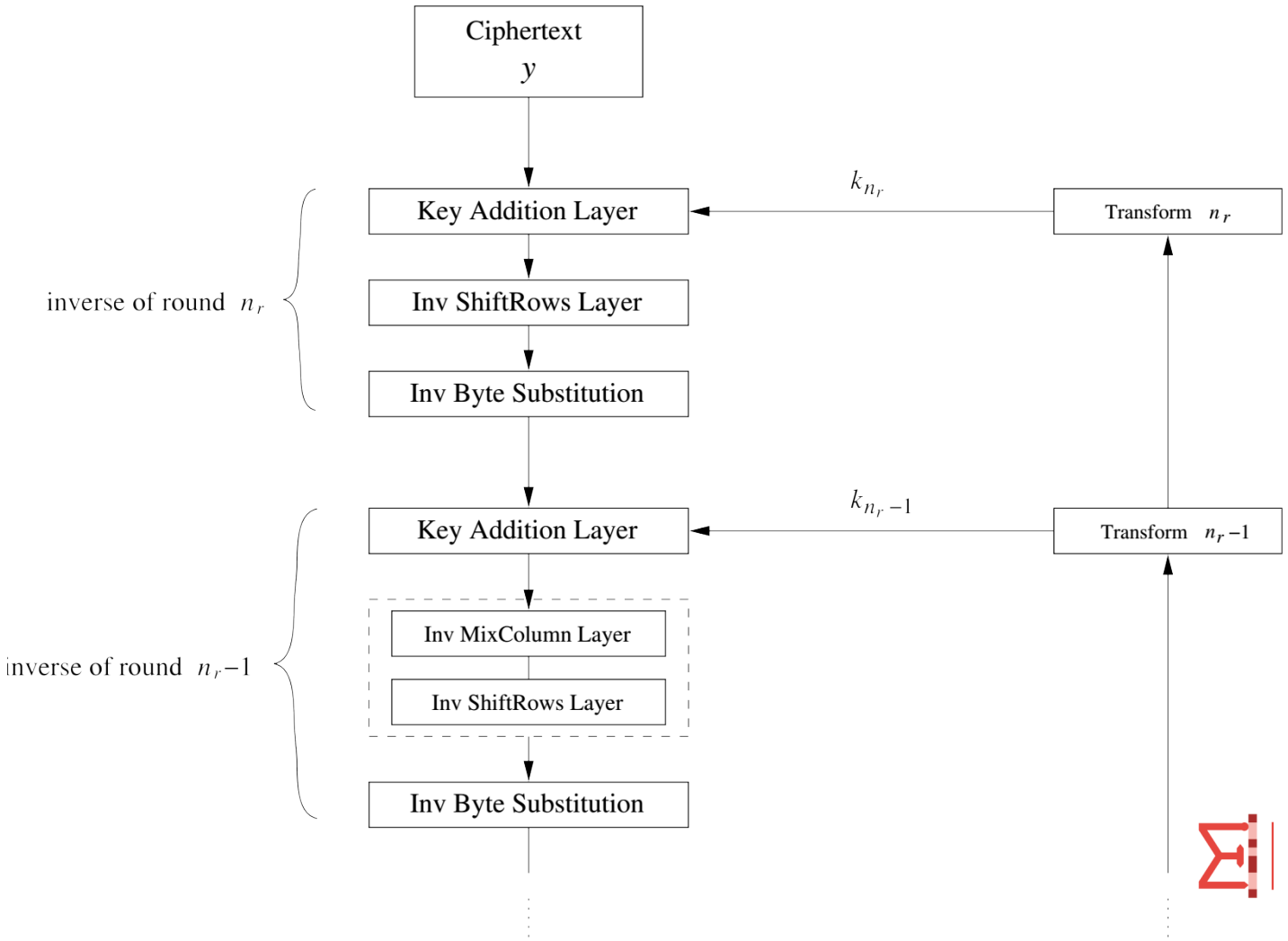
AES decryption reverses the transformations used during encryption.

Encryption		Decryption
SubBytes	$\longrightarrow$	**InvSubBytes**
ShiftRows	$\longrightarrow$	**InvShiftRows**
MixColumns	$\longrightarrow$	**InvMixColumns**
AddRoundKey	$\longrightarrow$	**AddRoundKey**

- **InvSubBytes** uses the inverse AES S-Box.
- **InvShiftRows** shifts the rows in the opposite direction.
- **InvMixColumns** applies the inverse column transformation.
- **AddRoundKey** remains unchanged because XOR is self-inverse.

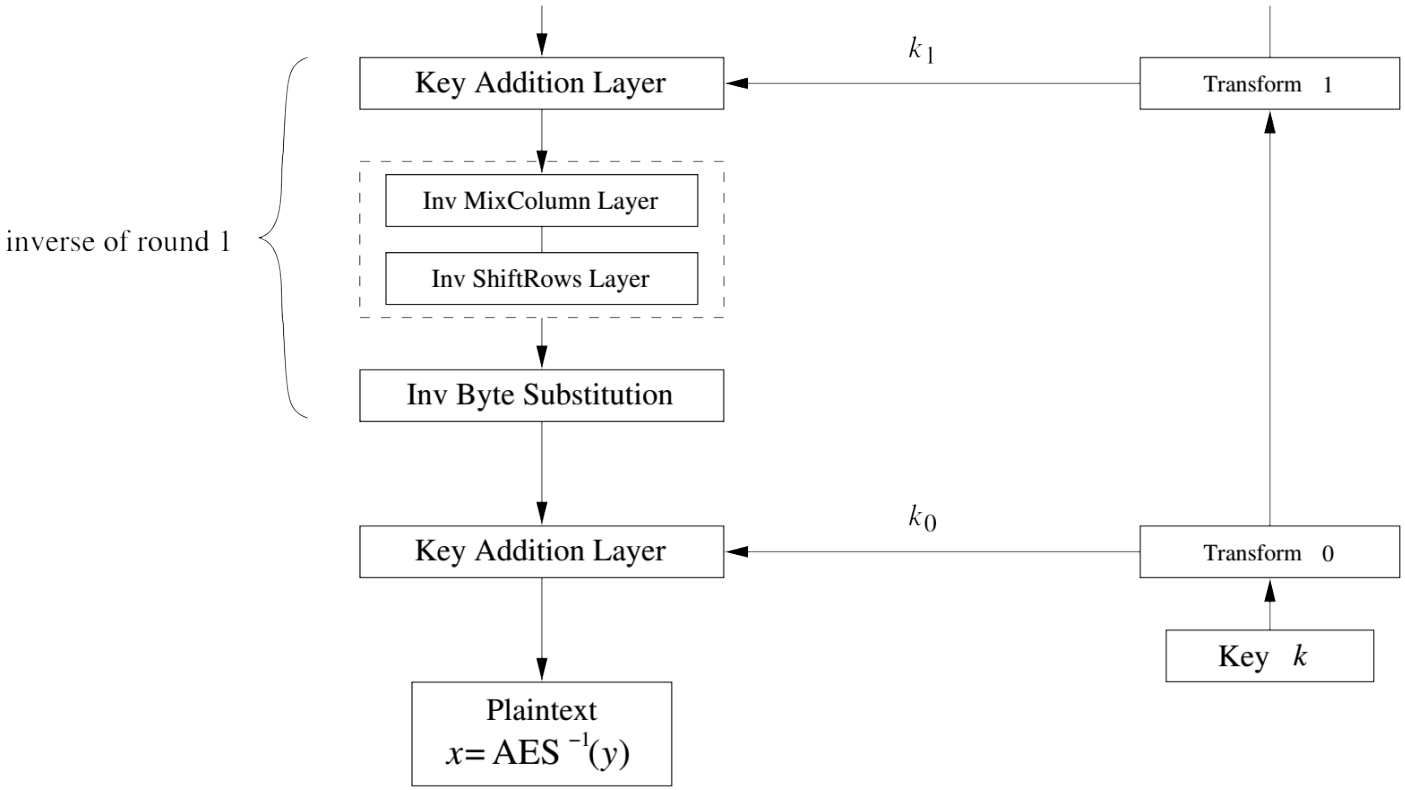
SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Decryption AES



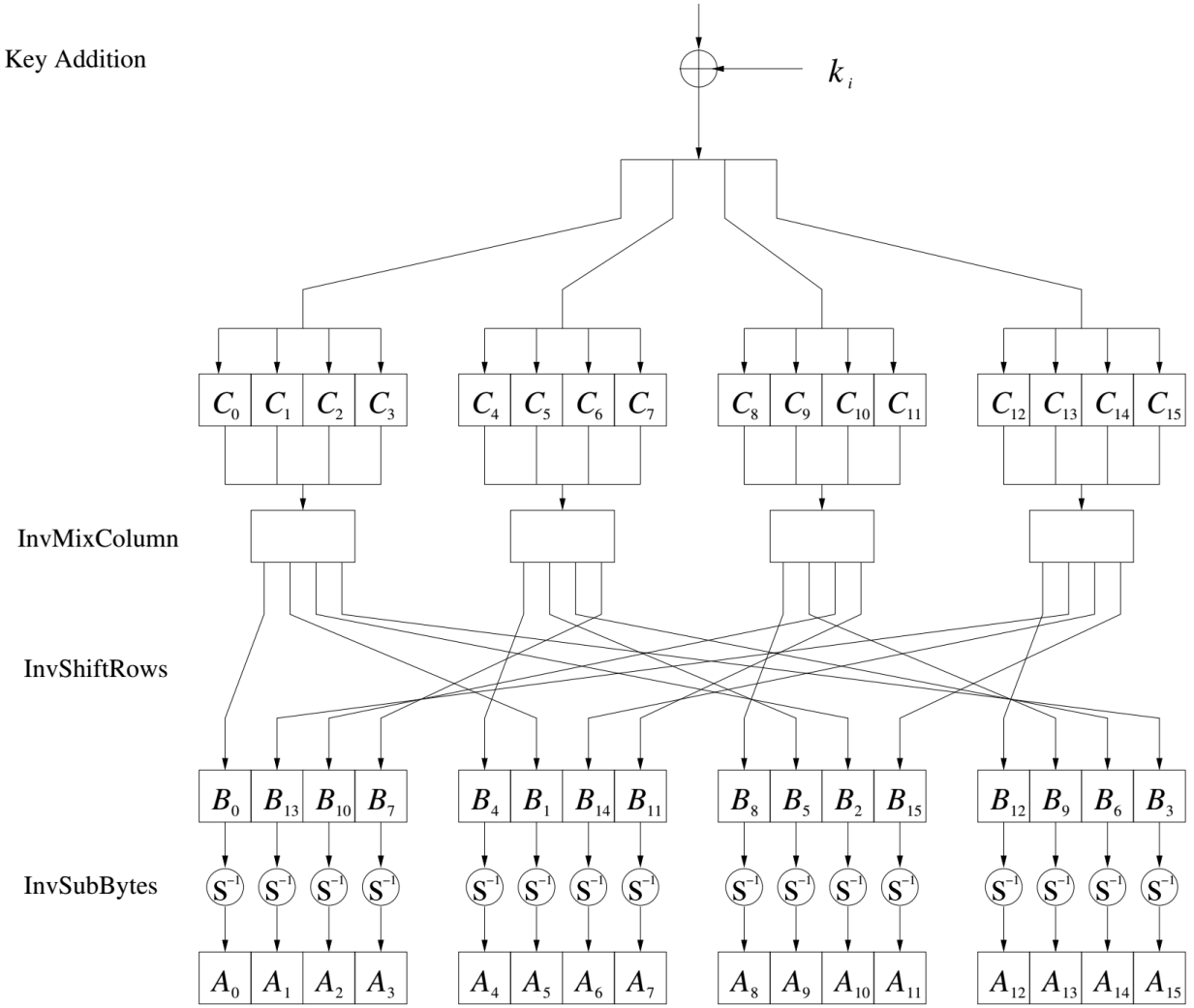
SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Decryption AES



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Decryption AES



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES Decryption

InvMixColumns Transformation

During AES decryption, each 4-byte column of the state is multiplied by the inverse MixColumns matrix.

$$\begin{bmatrix} B_0 \\ B_1 \\ B_2 \\ B_3 \end{bmatrix} = \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} C_0 \\ C_1 \\ C_2 \\ C_3 \end{bmatrix}$$

- Each input column contains **4 bytes**.
- The constants

  $$09,\ 0B,\ 0D,\ 0E$$

  are fixed elements of $\mathrm{GF}(2^8)$.
- All multiplications and additions are performed in $\mathrm{GF}(2^8)$.
- Addition in $\mathrm{GF}(2^8)$ is implemented using XOR.

InvMixColumns reverses the diffusion introduced by MixColumns.



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES Decryption

InvMixColumns Constants in $\mathrm{GF}(2^8)$

$$\begin{pmatrix} B_4 \\ B_5 \\ B_6 \\ B_7 \end{pmatrix} = \begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix} \begin{pmatrix} C_4 \\ C_5 \\ C_6 \\ C_7 \end{pmatrix}$$

Each state byte B_i and C_i, as well as each matrix constant, is an element of

$$\mathrm{GF}(2^8).$$

The constants used in InvMixColumns are

$$09, \quad 0B, \quad 0D, \quad 0E.$$

They are written in hexadecimal notation and interpreted as binary polynomials.

For example,

$$(0B)_{16} = (00001011)_2$$

corresponds to the polynomial

$$\boxed{(0B)_{16} \longleftrightarrow x^3 + x + 1}$$

because

$$00001011 = x^3 + x + 1.$$

AND COMPUTATIONAL SCIENCES

AES Decryption

InvShiftRows Transformation

The **InvShiftRows** transformation reverses the byte permutation introduced by ShiftRows.

B_0	B_4	B_8	B_{12}
B_1	B_5	B_9	B_{13}
B_2	B_6	B_{10}	B_{14}
B_3	B_7	B_{11}	B_{15}

the inverse ShiftRows sublayer yields the output:

B_0	B_4	B_8	B_{12}	no shift
B_{13}	B_1	B_5	B_9	⟵ three positions left shift
B_{10}	B_{14}	B_2	B_6	⟵ two positions left shift
B_7	B_{11}	B_{15}	B_3	⟵ one position left shift

Each row is cyclically shifted to the **right**:

- **Row 0:** no shift
- **Row 1:** shift right by 1 byte
- **Row 2:** shift right by 2 bytes
- **Row 3:** shift right by 3 bytes

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES Decryption

InvSubBytes Transformation

The **InvSubBytes** transformation reverses the nonlinear substitution performed by SubBytes during encryption.

For each state byte B_i, the inverse S-Box is applied:

$$\boxed{A_i = S^{-1}(B_i)}$$

Each byte is processed independently using the same **inverse AES S-Box**.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES Decryption

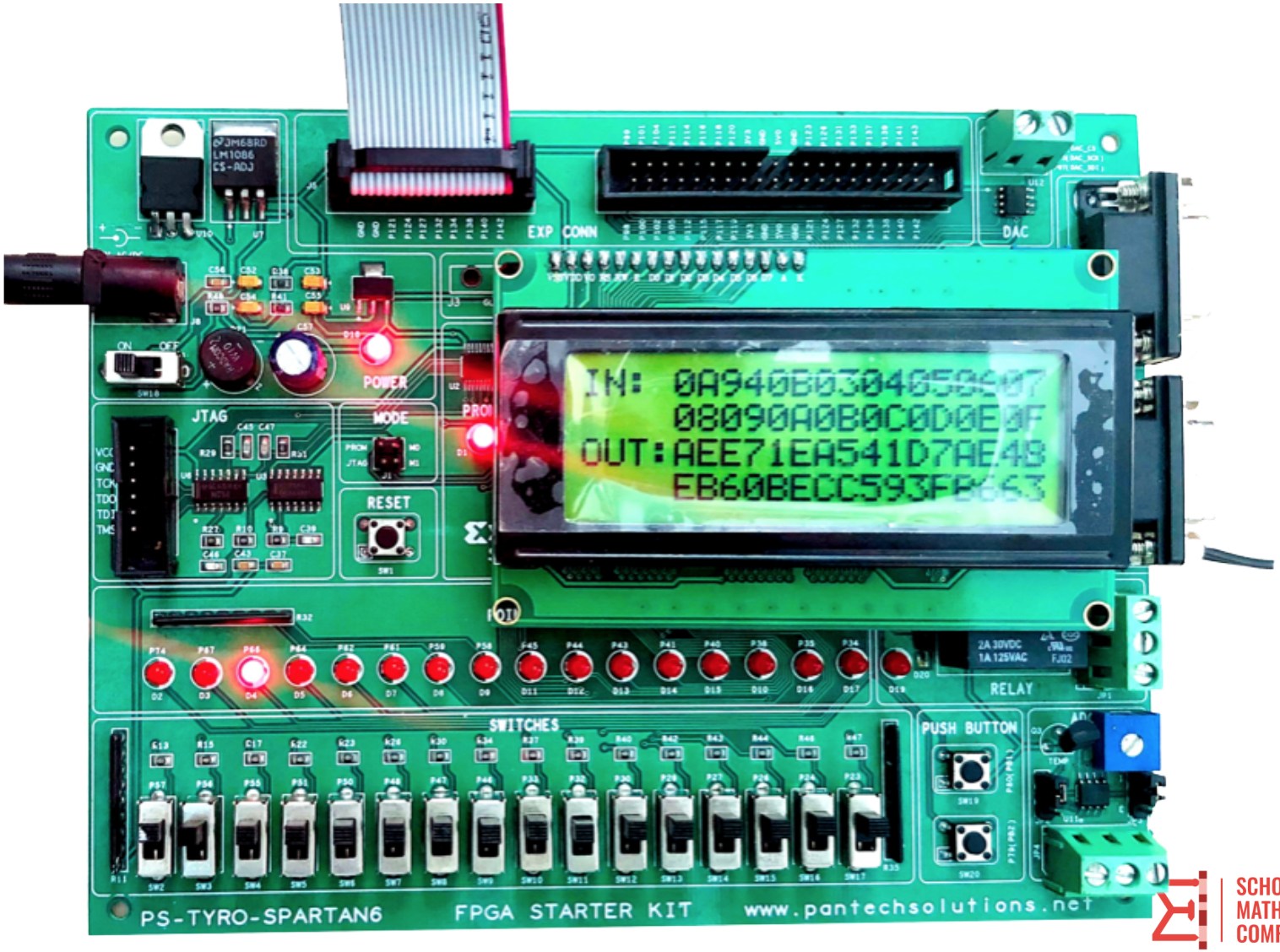
Inverse AES S-Box: Substitution values in hexadecimal notation for input byte (xy)

		y														
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

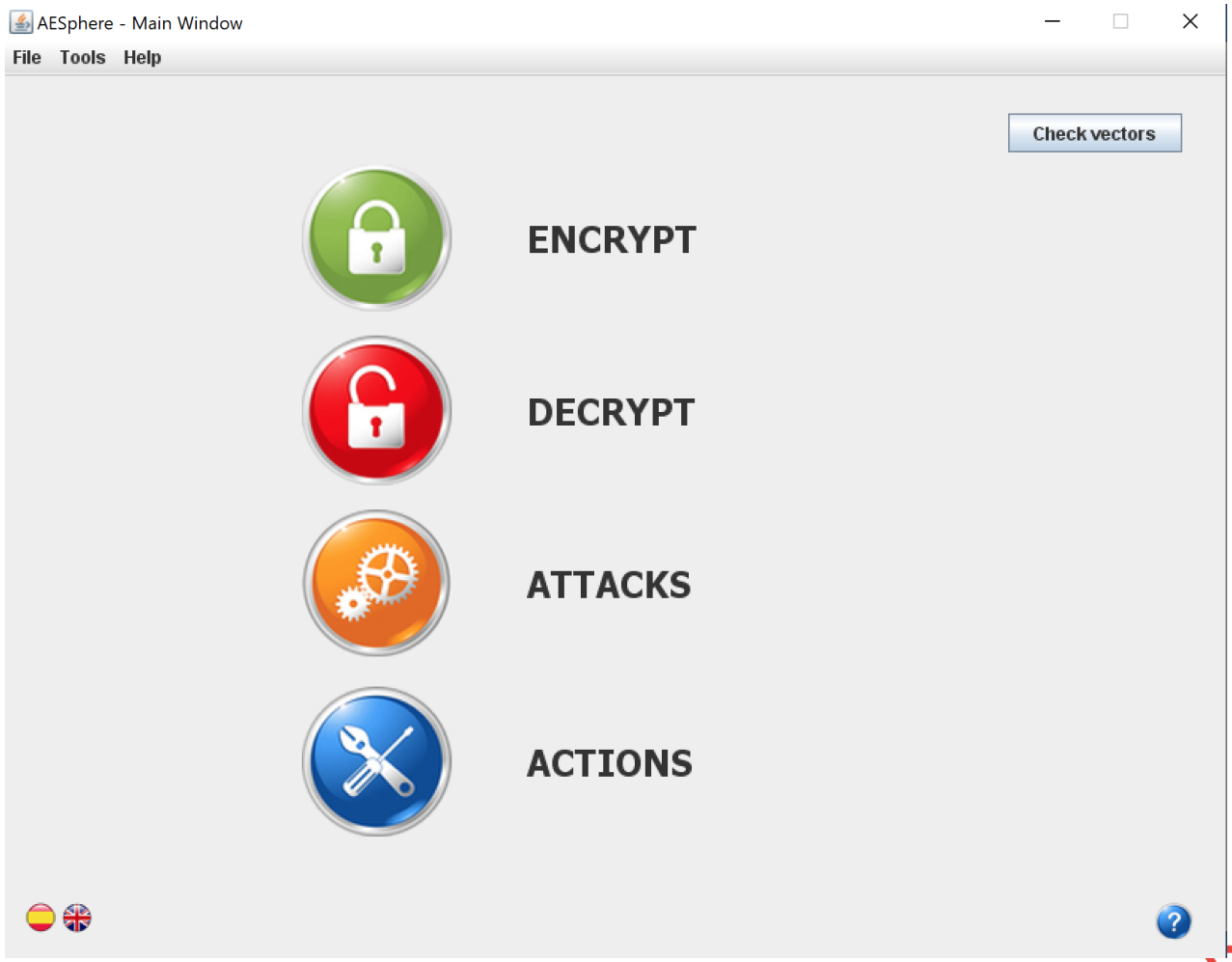
Implementation & Modes of Operation

Implementation in Software and Hardware



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

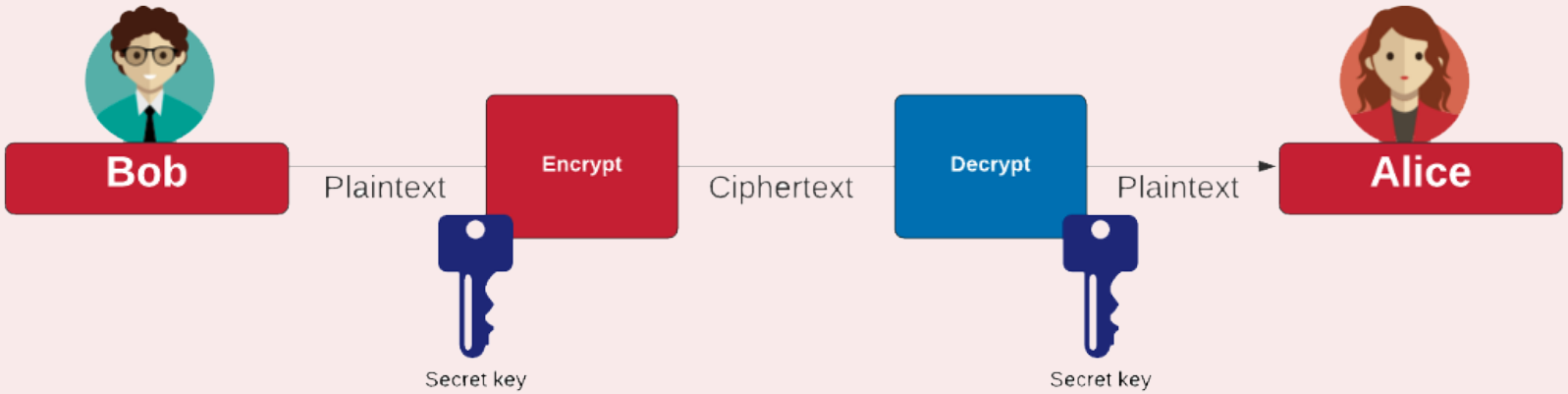
Implementation in Software and Hardware



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Cryptography

Overview of Symmetric key



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Modes of Operation

Why Do We Need Modes of Operation?

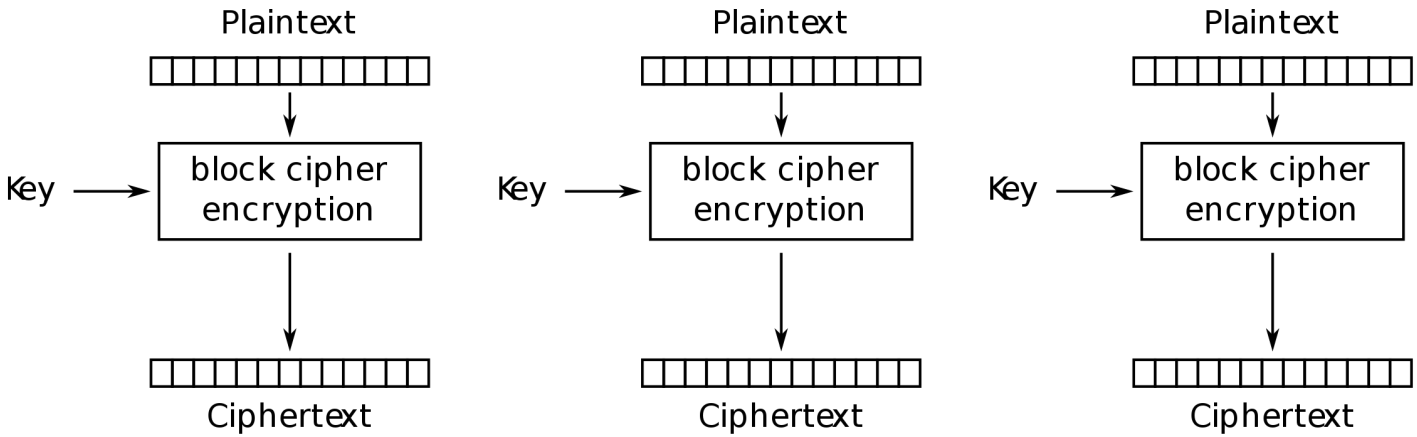
A block cipher such as AES encrypts only a **fixed-size block**. A **mode of operation** defines how the cipher is applied to messages containing multiple blocks.

Five classical modes of operation are:

1. **Electronic Codebook (ECB)**
2. **Cipher Block Chaining (CBC)**
3. **Cipher Feedback (CFB)**
4. **Output Feedback (OFB)**
5. **Counter (CTR)**

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Modes of Operation: Electronic Codebook (ECB)



Electronic Codebook (ECB) mode encryption

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Electronic Codebook (ECB)

Main Weakness of ECB

- Each plaintext block is encrypted **independently**:

$$C_i = E_K(P_i)$$

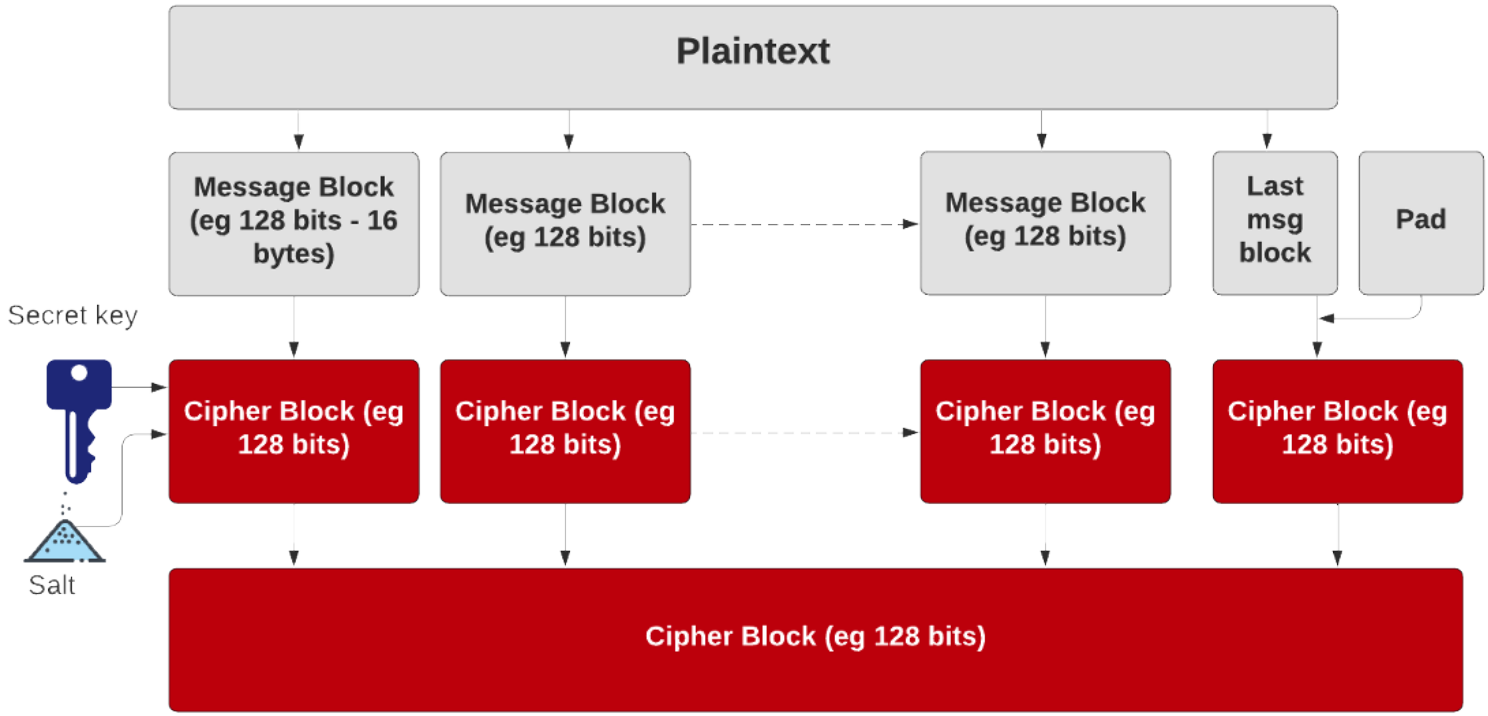
- Therefore, identical plaintext blocks produce **identical ciphertext blocks**:

$$P_i = P_j \implies C_i = C_j$$

- Repeated patterns in the plaintext may remain visible in the ciphertext.
- For this reason, ECB is **not recommended for long or structured messages**.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Modes of Operation: Electronic Codebook (ECB)



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Cipher Block Chaining (CBC)

How CBC Works

- Before encryption, each plaintext block is XORed with the **previous ciphertext block**.

$$C_i = E_K(P_i \oplus C_{i-1})$$

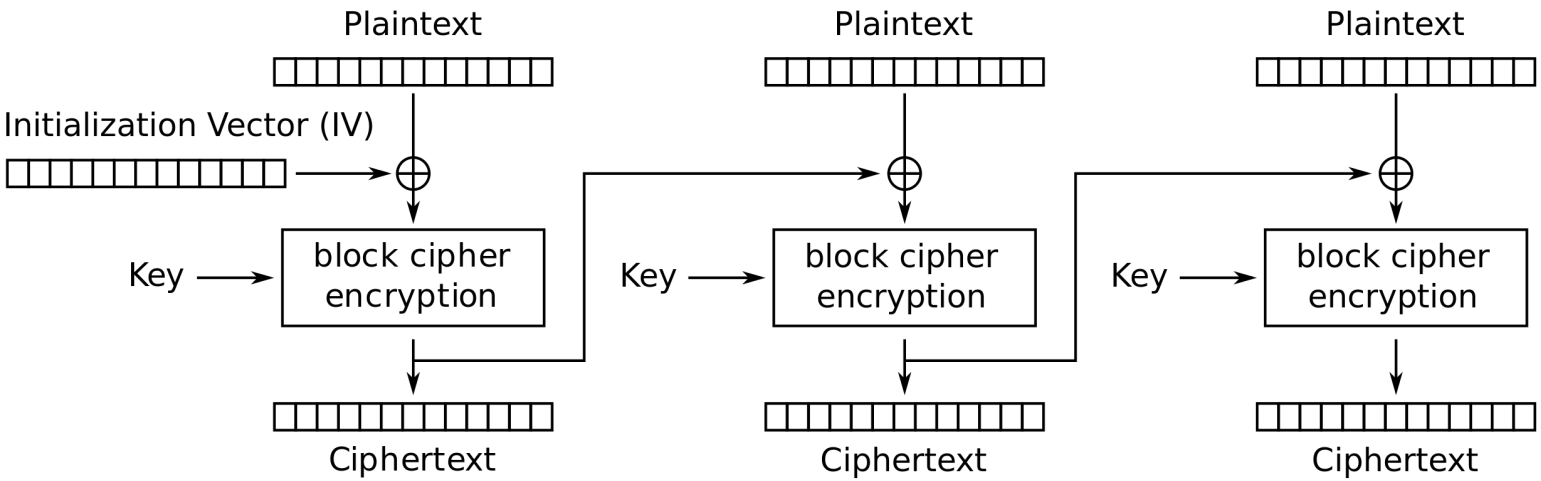
- For the first block, an **Initialization Vector (IV)** is used:

$$C_1 = E_K(P_1 \oplus IV)$$

- Each ciphertext block therefore depends on the current plaintext block and on the previous ciphertext block.
- Unlike ECB, identical plaintext blocks do not necessarily produce identical ciphertext blocks.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

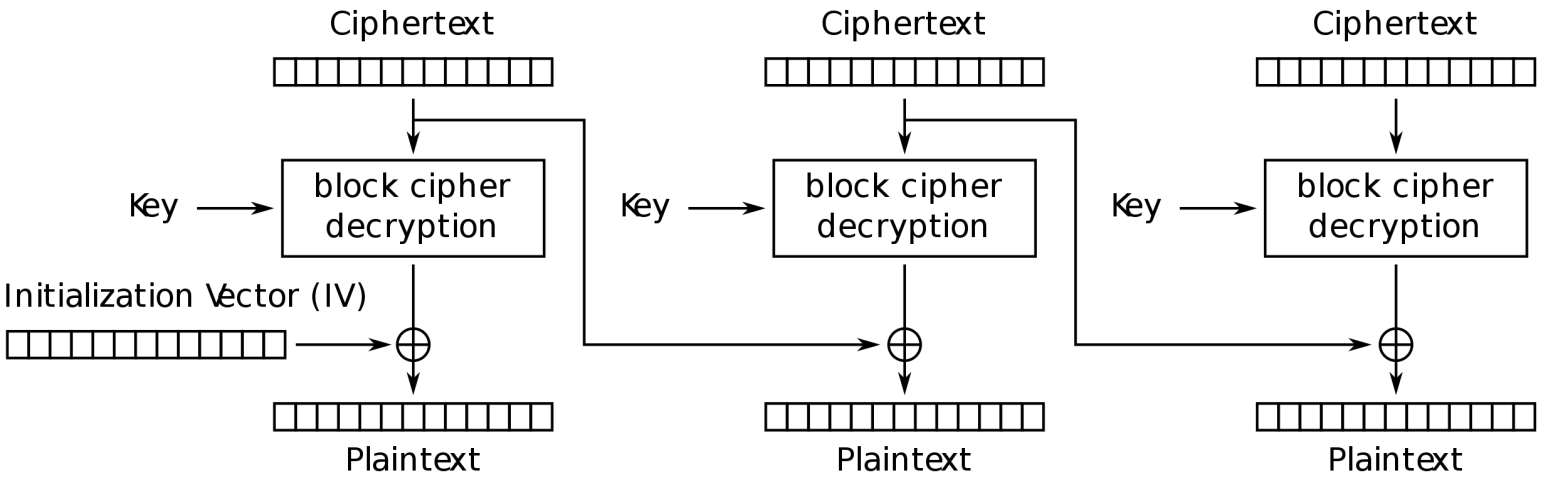
Modes of Operation: Cipher Block Chaining (CBC)



Cipher Block Chaining (CBC) mode encryption

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Modes of Operation: Cipher Block Chaining (CBC)



Cipher Block Chaining (CBC) mode decryption

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Modes of Operation

Cipher Block Chaining (CBC)

- CBC uses an **Initialization Vector (IV)** for the first plaintext block and the previous ciphertext block for the following blocks.
- Therefore, identical plaintext blocks generally produce **different ciphertext blocks** because they are XORed with different values before encryption:

$$C_i = E_K(P_i \oplus C_{i-1})$$

- This chaining mechanism helps to **hide repetitive patterns** in the plaintext.
- As a result, CBC provides better pattern protection than **ECB mode**.

Cryptography

Modes of Operation: Cipher Block Chaining (CBC)

- Cipher block chaining (CBC) mode a technique in which the same plaintext block, if repeated, produces different ciphertext blocks.
- It is used to achieve confidentiality, the CBC mode can be used for authentication.

Cryptography

Modes of Operation: Cipher Block Chaining (CBC)

- Encryption Algorithm
  - $C_1 = E(K, [P_1 \oplus IV])$
  - $C_j = E(K, [P_j \oplus C_{j-1}])$ $\quad j = 2, ...N$
- Decryption Algorithm
  - $P_1 = D(K, C_1) \oplus IV$
  - $P_j = D(K, C_j) \oplus C_{j-1}$ $\quad j = 2, ...N$

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Cryptography

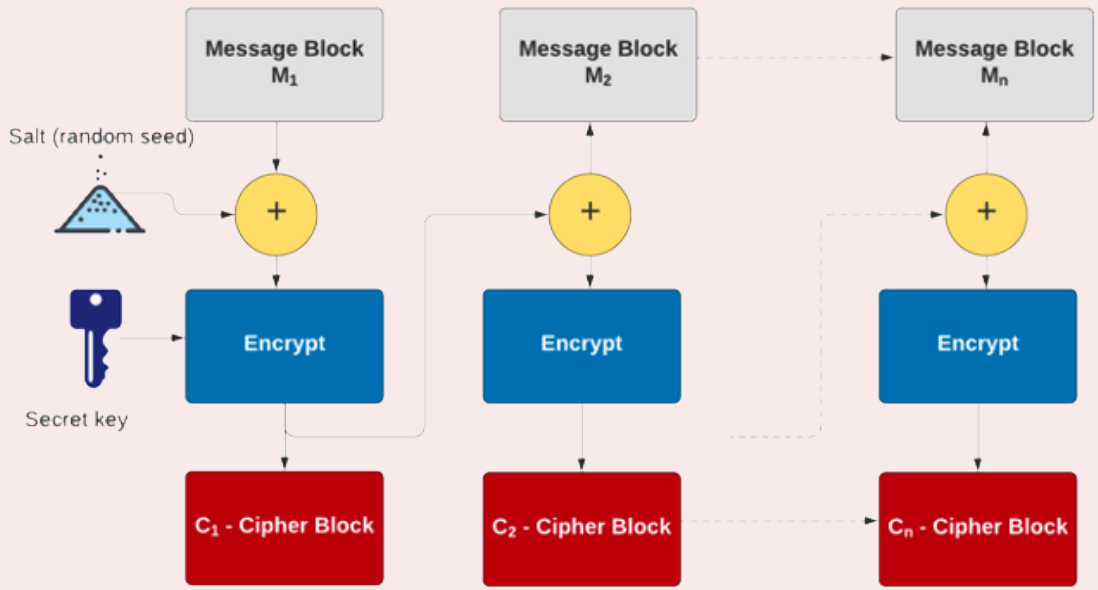
Modes of Operation: Cipher Block Chaining (CBC)

- A ciphertext block depends on all blocks before it.
- Any change to a block affects all following ciphertext blocks
- Initialization Vector (IV) must be known to sender & receiver
- Initialization Vector (IV) must be sent encrypted in CBC mode before rest of message

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

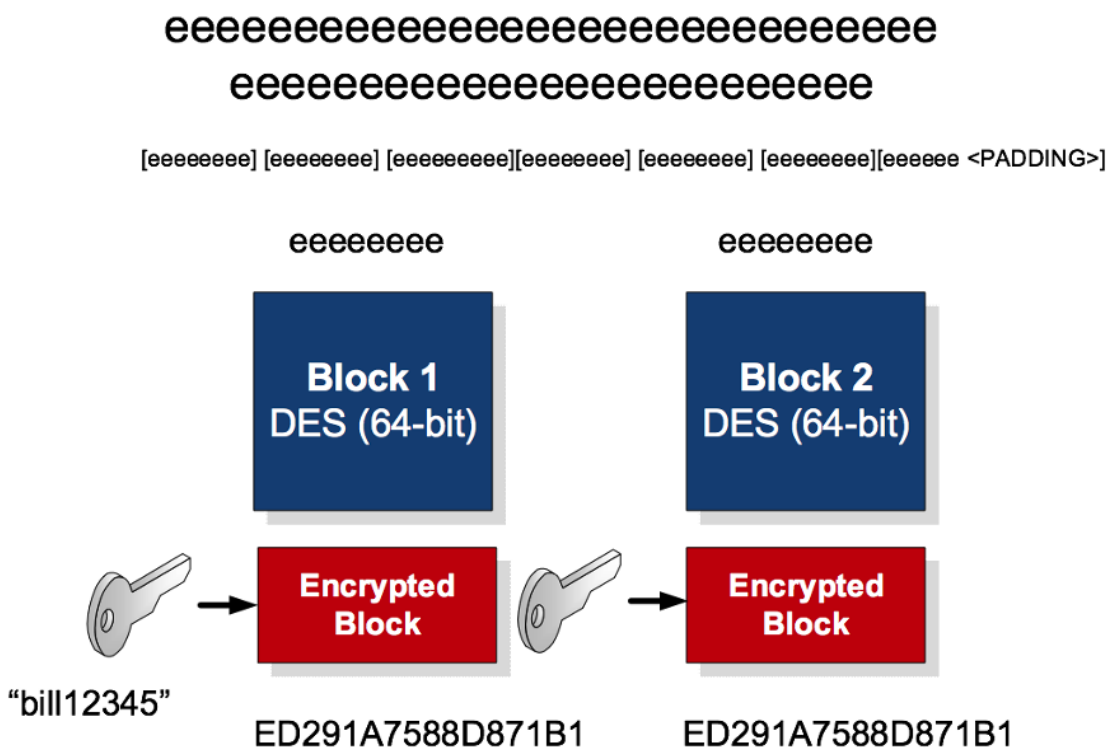
Cryptography

Modes of Operation: Cipher Block Chaining (CBC)



SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

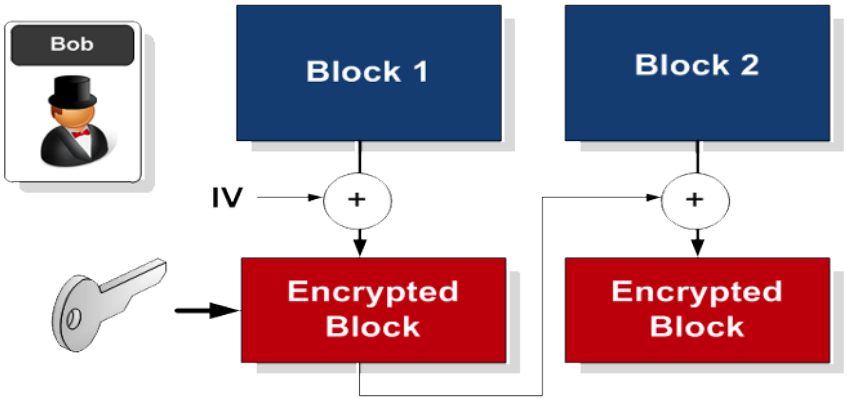
AES using ECB vs AES using CBC)



ED291A7588D871B1ED291A7588D871B1ED291A7588D871B1ED291A7588D871B1ED291A7588D871B1ED291A7588D871B18D6DF6795DDEDACD

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

AES using ECB vs AES using CBC)



Cipher Block Chaining (CBC). This method uses the IV for the first block, and then the results from the previous block to encrypt the current block.



Original image

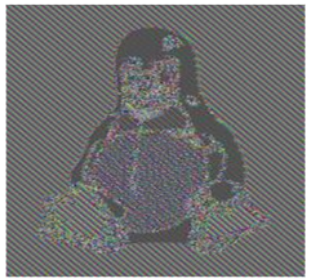


Image with AES using ECB

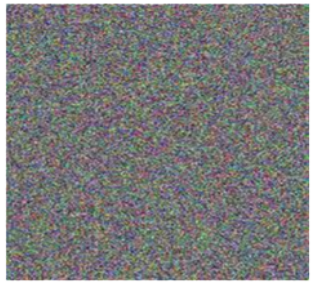


Image with AES using CBC

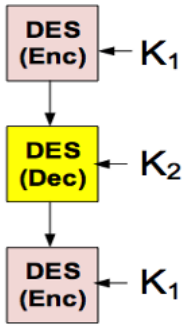
SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Cryptography

DES

DES. DES encryption algorithm is block cipher and uses a 64-bit block and a 64-bit encryption key.

3DES. DES encryption algorithm is block cipher and uses a 64-bit block and a 64-bit encryption key (of which only 56 bits are actively used in the encryption process). Unfortunately DES has been around for a long time, and the 56-bit version is now easily crackable (in less than a day, on fairly modest equipment). An enhancement, and one which is still fairly compatible with DES, is the 3-DES algorithm. It has three phases, and splits the key into two. Overall the key size is typically 112 bits (2x54 bits - with a combination of the three keys - of which two of the keys are typically the same). The algorithm is EncryptK3(DecryptK2(EncryptK1(message), where K1 and K3 are typically the same (to keep compatibility).



Twofish

Bruce Schneier created Twofish with a general-purpose private key block cipher encryption algorithm.

AES

AES. AES (or Rijndael) is a new block cipher, and is the new replacement for DES, and uses 128-bit blocks with 128, 192 and 256 bit encryption keys. It was selected by NIST in 2001 (after a five year standardisation process). The name Rijndael comes from its Belgium creators: Joan Daemen and Vincent Rijmen.

RC2

RC2. RC2. RC2 ("Rivest Cipher") is a block cipher, and is seen as a replacement for DES. It was created by Ron Rivest in 1987, and is a 64-bit block code and can have a key size from 40 bits to 128-bits (in increments of 8 bits). The 40-bit key version is seen as weak, as the encryption key is so small, but is favoured by governments for export purposes, as it can be easily cracked. In this case the key is created from a Key and an IV (Initialisation Vector). The key has 12 characters (96 bits), and the IV has 8 characters (64 bits), which go to make the overall key.

Blowfish

Blowfish. Bruce Schneier created Blowfish with a general-purpose private key block cipher encryption algorithm.
Blowfish (with CBC). Blowfishcbc. With CBC we split the message into blocks and encrypt each block. The input from the first stage is the IV (Initialisation Vector), and the input to the following stages is the output from the previous stage. In this example we will use Blowfish to encrypt, using CBC.

Others

- Skipjack. Skip jack. Skipjack is a block cipher, using private-key encryption algorithm, and designed by NSA.
- Camellia. Camillia is a block cipher created by Mitsubishi and NTT.
- RC4. RC4 is a stream cipher used in WEP (in wireless encryption).
- Affine. Affine is a stream cipher which uses an equation to encrypt.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Cryptography (cont.)

Laboratory 3: AES Implementation

- Implement the **Advanced Encryption Standard (AES)** directly from the specification.
- The implementation must support the three standard key sizes:

AES Version	Key Size	Rounds
AES-128	128 bits	10
AES-192	192 bits	12
AES-256	256 bits	14

- Implement the main AES transformations: **SubBytes**, **ShiftRows**, **MixColumns**, and **AddRoundKey**.
- Implement the corresponding **Key Expansion** for 128-, 192-, and 256-bit keys.
- Implement and verify both **encryption and decryption**.
- Validate the implementation using **official AES test vectors**.

Cryptography (cont.)

Implementation Requirement

The AES cryptographic core must be implemented by the students; **ready-made AES libraries are not allowed**.

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

Cryptography

Advanced Encryption Standard (AES)

Cristhian Iza

School of Mathematical and Computational Sciences
Yachay Tech University

September 9, 2026

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES